RAJALAKSHMI ENGINEERING COLLEGE
RAJALAKSHMI NAGAR, THANDALAM – 602 105

RAJALAKSHMI ENGINEERING COLLEGE

CS23334 – Fundamentals of Data Science Lab

Laboratory Record Note Book

Name : DHAARANI. K

Year / Branch / Section : II - CSE - I

University Register No. : 2116250701150

College Roll No. : 250701150

Semester : III

Academic Year : 2026 - 2027

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23334 – Fundamentals of Data Science Lab

NAME	DHAARANI K
ROLL NO	250701150
YEAR	II
SEC	I

RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: DHAARANI. K

Academic Year: 2026-27 **Semester:** III **Branch:** CSE

Register No. 250701150

Certified that this is the bonafide record of work done by the above student in the Fundamentals of Data Science ***Laboratory during the academic year 2026 – 2027.***

Signature of Faculty in-charge

Submitted for the Practical Examination held on .....

Internal Examiner

External Examiner

INDEX

Sl.NO	Experiment	Date	GitHub
1.a	Analyze the trend of data science job postings over the last decade for the given data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib /seaborn for visualization.	29/07/26	
1.b	Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.	29/07/26	
1.c	Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.	05/08/26	
1.d	Conduct an experiment to encrypt and decrypt given sensitive data Use the cryptography library fernet.	05/08/26	
2	Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.	12/08/26	
3.a	Conduct an experiment to show data visualization using line plot	19/08/26	
3.b	Conduct an experiment to show data visualization using bar chart	19/08/26	
3.c	Conduct an experiment to show data visualization using pie chart	19/08/26	
4	To Count the frequency of occurrence of a word in a body of text is often needed during text processing.	26/08/26	
5	Data Collection and Initial Exploration	16/09/26	
6	Time Series Analysis of Website Traffic	16/09/26	

7	Random Sampling and Sampling Distribution	23/09/26	
8	Conduct Z test for the Data given using Python	23/09/26	
9	Experiment to understand Matplotlib library use cases in Data Science through visualization.	23/09/26	
10	Experiment to understand array function in Data science.	23/09/26	
11	Experiment to understand pandas library use cases in Data science.	30/09/26	
12	Experiment to detect outliers in a given data set.	30/09/26	
13	Experiment to handle missing values and inappropriate data in a given data set.	30/09/26	
14	Experiment to understand feature scaling	30/09/26	
15	Experiment to understand the data preprocessing in Data science	30/09/26	
16	Experiment to understand EDA-Quantitative and Qualitative analysis.	07/10/26	
17	Experiment to understand Linear Regression for a given data set.	07/10/26	
18	Experiment to understand KNN algorithm for a given data set.	07/10/26	
19	Experiment to understand Logistic Regression for a given data set.	07/10/26	
20	Experiment to understand K-means clustering algorithm for a given data set.	07/10/26	

Exp No:1.a Analyze the trend of data science job postings over the last decade

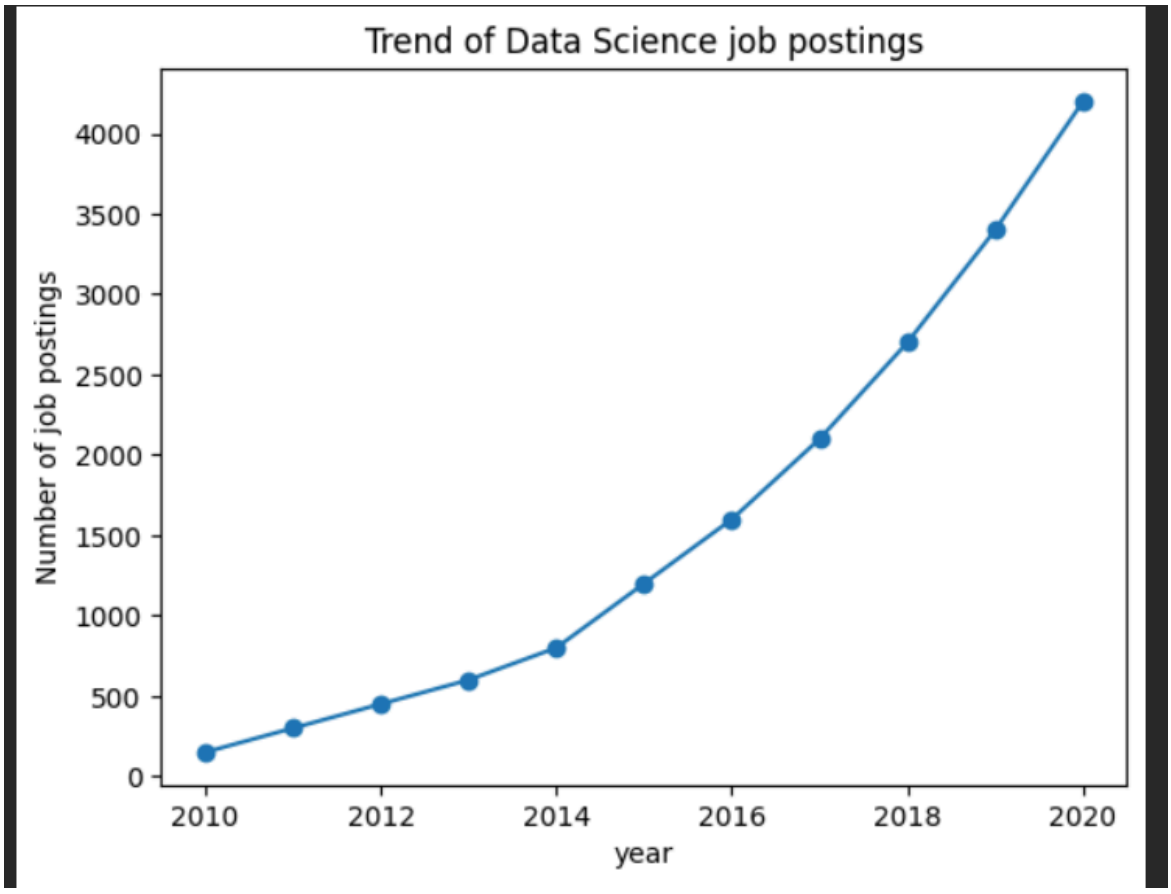
Aim: To analyze the trend of data science job postings over the last decade

Code :

```
        import pandas as pd
import matplotlib.pyplot as plt
data = {'Year': list(range(2010, 2021)),
'Job Postings': [150, 300, 450, 600, 800, 1200, 1600, 2100, 2700, 3400, 4200]}

df = pd.DataFrame(data)
plt.plot(df['Year'], df['Job Postings'], marker='o')
plt.title('Trend of Data Science Job Postings')
plt.xlabel('Year')
plt.ylabel('Number ofJob Postings')
plt.show()
```

Output:



Result:

Thus,the given data was analyzed successfully and required output was obtained.

Exp No:1.b Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.

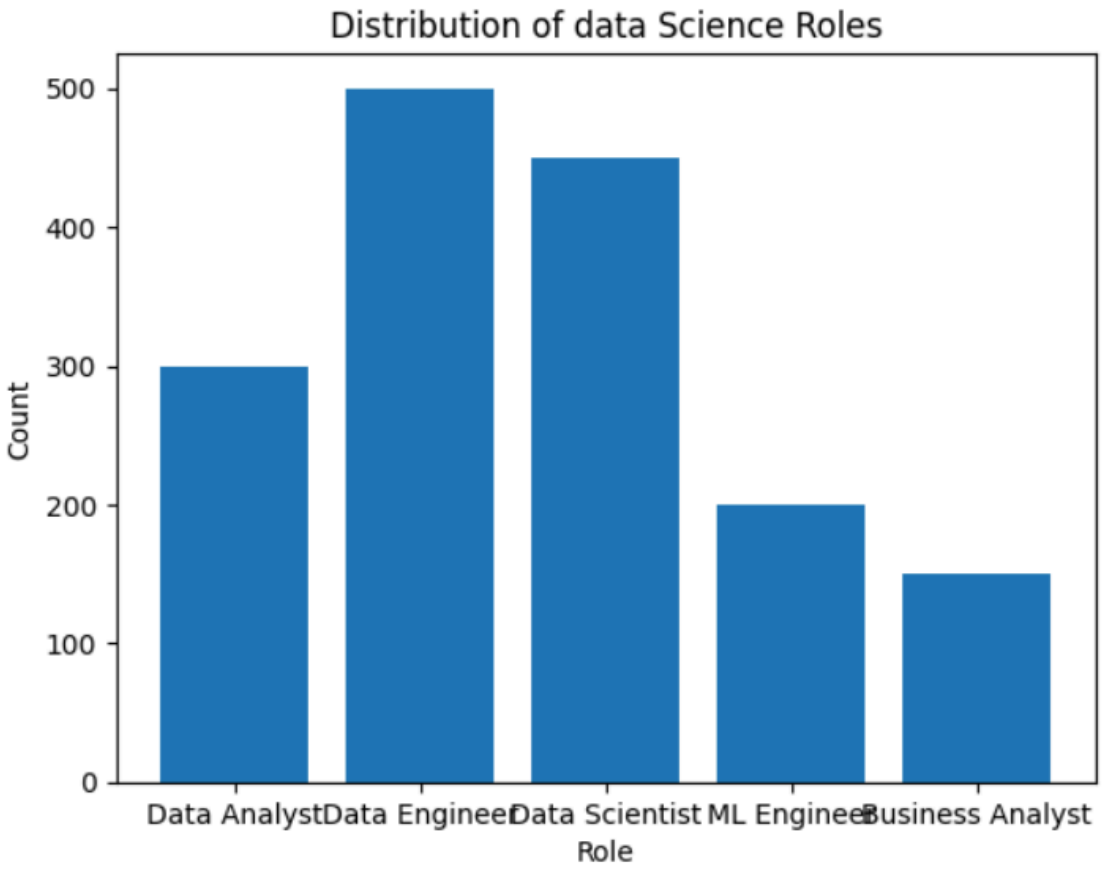
Aim:

To analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.

Code:

```
roles = ['Data Analyst', 'Data Engineer', 'Data Scientist', 'ML Engineer',
'Business Analyst']
counts = [300, 500, 450, 200, 150]
plt.bar(roles, counts)
plt.title('Distribution of Data Science Roles')
plt.xlabel('Role')
plt.ylabel('Count')
plt.show()
```

Output:



Result:

Thus,the given data was analyzed successfully and required output was obtained.

Exp No:1.c Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.

Aim:

To conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.

Code:

```
# Structured data example
structured_data = pd.DataFrame({
'ID': [1, 2, 3],
'Name': ['Alice', 'Bob', 'Charlie'],
'Age': [25, 30, 35]
})
print("Structured Data:\n", structured_data)

# Unstructured data example
unstructured_data = "This is an example of unstructured data. It can be a piece of text, an image,
or a video file."
print("\nUnstructured Data:\n", unstructured_data)

# Semi-structured data example (JSON)
semi_structured_data = {'ID': 1, 'Name': 'Alice', 'Attributes': {'Height': 165, 'Weight': 68}}
print("\nSemi-structured Data:\n", semi_structured_data)
```

Output:

```
Structured Data:
    ID     Name  Age
0   1    Alice   25
1   2      Bob   30
2   3  Cahrlie   35

Unstructured Data:
 his is an example of unstructured data.It can be a piece of text,an image or a video file.

Semi-structured Data:
 {'ID': 1, 'Name': 'Alice', 'Attributes': {'height': 165, 'Weight': 68}}
```

Result:

Thus,the given data was analyzed successfully and required output was obtained.

Exp No: 1.d Conduct an experiment to encrypt and decrypt given sensitive data.

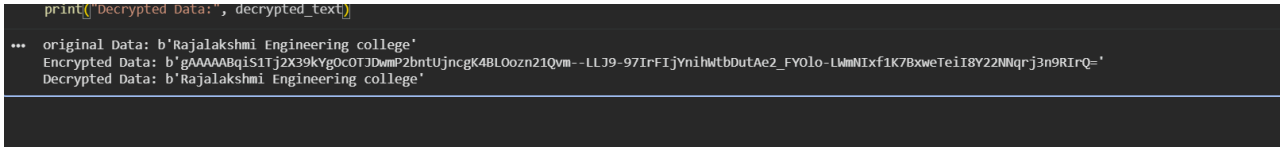
Aim:

To conduct an experiment to encrypt and decrypt given sensitive data.

Code:

```
# Generate key and encrypt data
from cryptography.fernet import Fernet
key = Fernet.generate_key()
f = Fernet(key)
token = f.encrypt(b"Rajalakshmi Engineering College")
token
b'...'
f.decrypt(token)
b'Rajalakshmi Engineering College'
key = Fernet.generate_key()
cipher_suite = Fernet(key)
plain_text = b"Rajalakshmi Engineering College."
cipher_text = cipher_suite.encrypt(plain_text)
# Decrypt data
decrypted_text = cipher_suite.decrypt(cipher_text)
print("Original Data:", plain_text)
print("Encrypted Data:", cipher_text)
print("Decrypted Data:", decrypted_text)
```

Output:



Result:

Thus,the given data was analyzed successfully and required output was obtained.

EXPERIMENT NO : 2

UPLOAD AND ANALYZE THE DATA SET GIVEN IN CSV FORMAT AND PERFORM DATA PREPROCESSING AND VISUALIZATION

AIM :

To upload and Analyze the data set given in csv format and perform data preprocessing and visualization.

CODE :

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
df=pd.read_csv('sales_data.csv')
print(df.head())
print(df.isnull().sum())
df['Sales']=df['Sales'].fillna(df['Sales'].mean())
df.dropna(subset=['Product','Quantity','Region'],inplace=True)
print(df.describe())
product_summary=df.groupby('Product')[['Sales','Quantity']].sum().reset_index()
print(product_summary)
plt.figure(figsize=(10,6))
plt.bar(product_summary['Product'],product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()
df['Date']=pd.to_datetime(df['Date'])
sales_over_time=df.groupby('Date')['Sales'].sum().reset_index()
```

```
plt.figure(figsize=(10,6))
plt.plot(sales_over_time['Date'],sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')
plt.show()
pivot_table=df.pivot_table(values='Sales',index='Region',columns='Product',aggfunc='sum',fill_value=0)
print(pivot_table)
correlation_matrix=df.corr(numeric_only=True)
print(correlation_matrix)
plt.figure(figsize=(8,6))
sns.heatmap(correlation_matrix,annot=True,cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

OUTPUT :

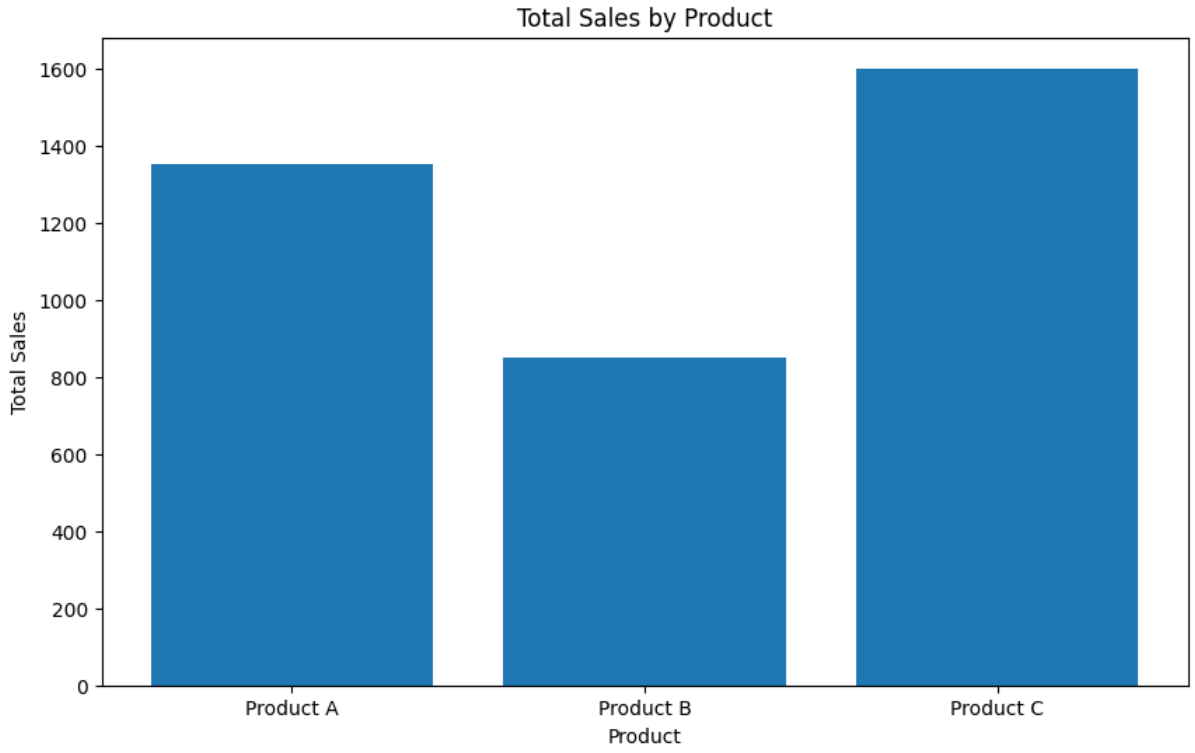
	Date	Product	Sales	Quantity	Region
0	2023-01-01	Product A	200	4	North
1	2023-01-02	Product B	150	3	South
2	2023-01-03	Product A	220	5	North
3	2023-01-04	Product C	300	6	East
4	2023-01-05	Product B	180	4	West

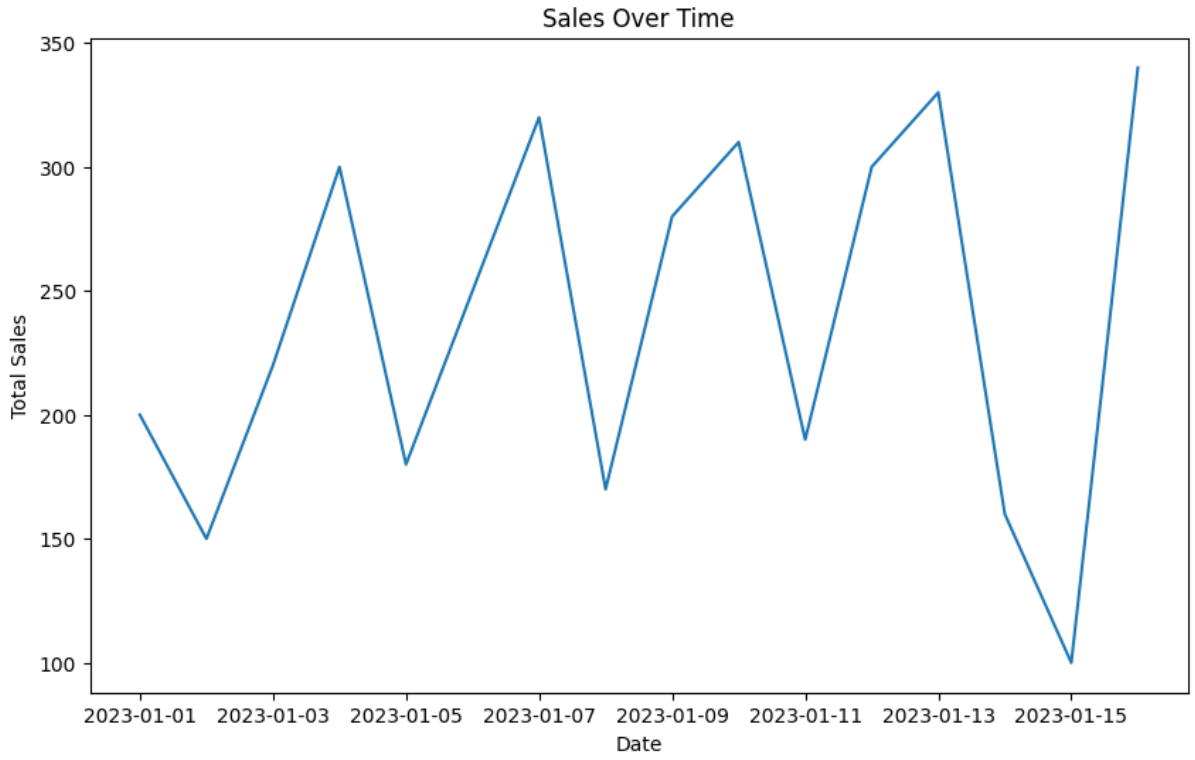
```
Date        0
Product     0
Sales       0
Quantity    0
Region      0
```

dtype: int64

	Sales	Quantity
count	16.00000	16.0000
mean	237.50000	5.4375
std	75.40999	1.7500
min	100.00000	3.0000
25%	177.50000	4.0000
50%	235.00000	5.5000
75%	302.50000	7.0000
max	340.00000	8.0000

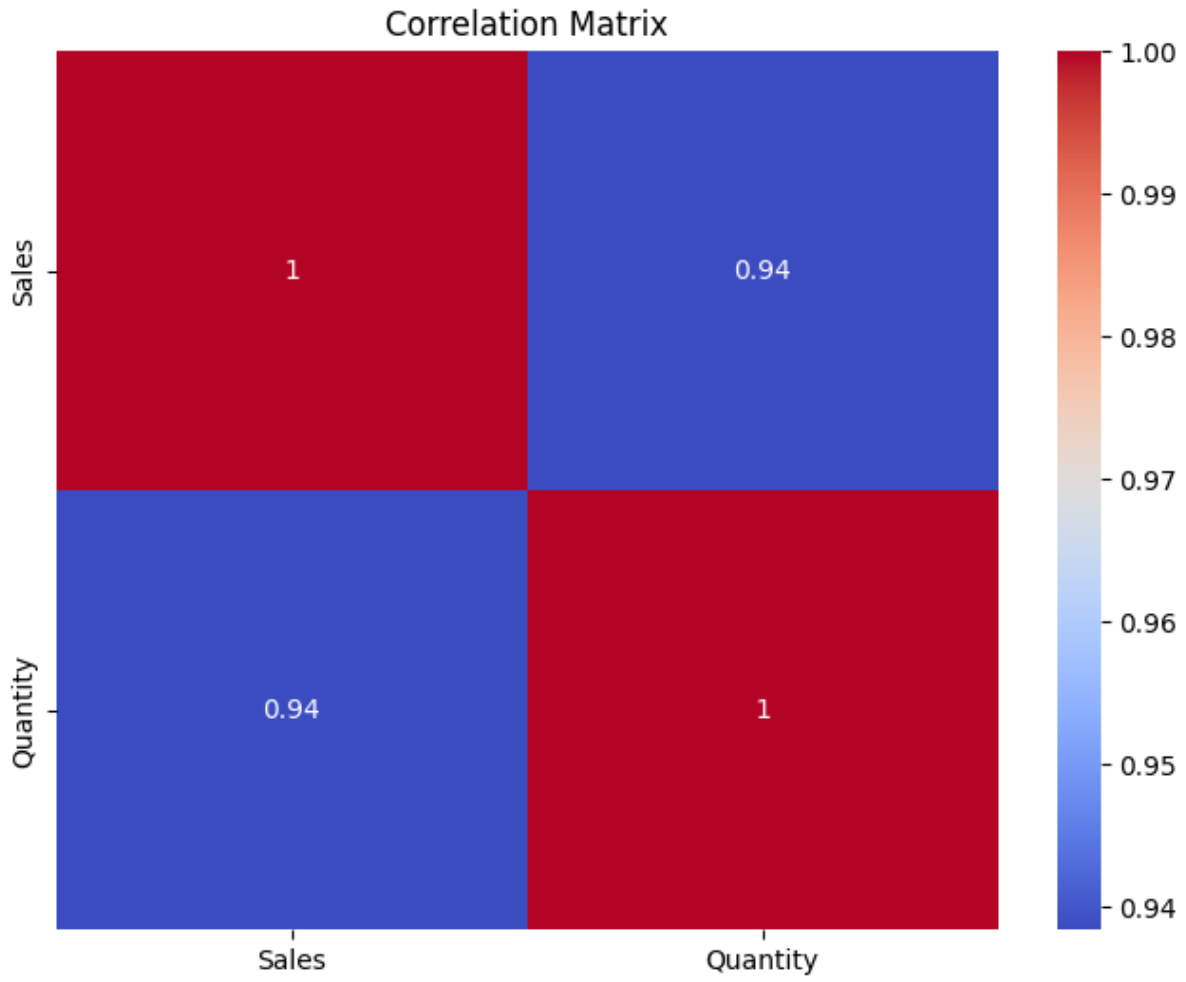
	Product	Sales	Quantity
0	Product A	1350	33
1	Product B	850	18
2	Product C	1600	36





Product	Product A	Product B	Product C
Region			
East	0	0	1600
North	1350	0	0
South	0	480	0
West	0	370	0

	Sales	Quantity
Sales	1.000000	0.938362
Quantity	0.938362	1.000000



RESULT :

Thus,the given data was analyzed successfully and required output was obtained.

Exp No:3.a Conduct an experiment to show data visualization using line plot

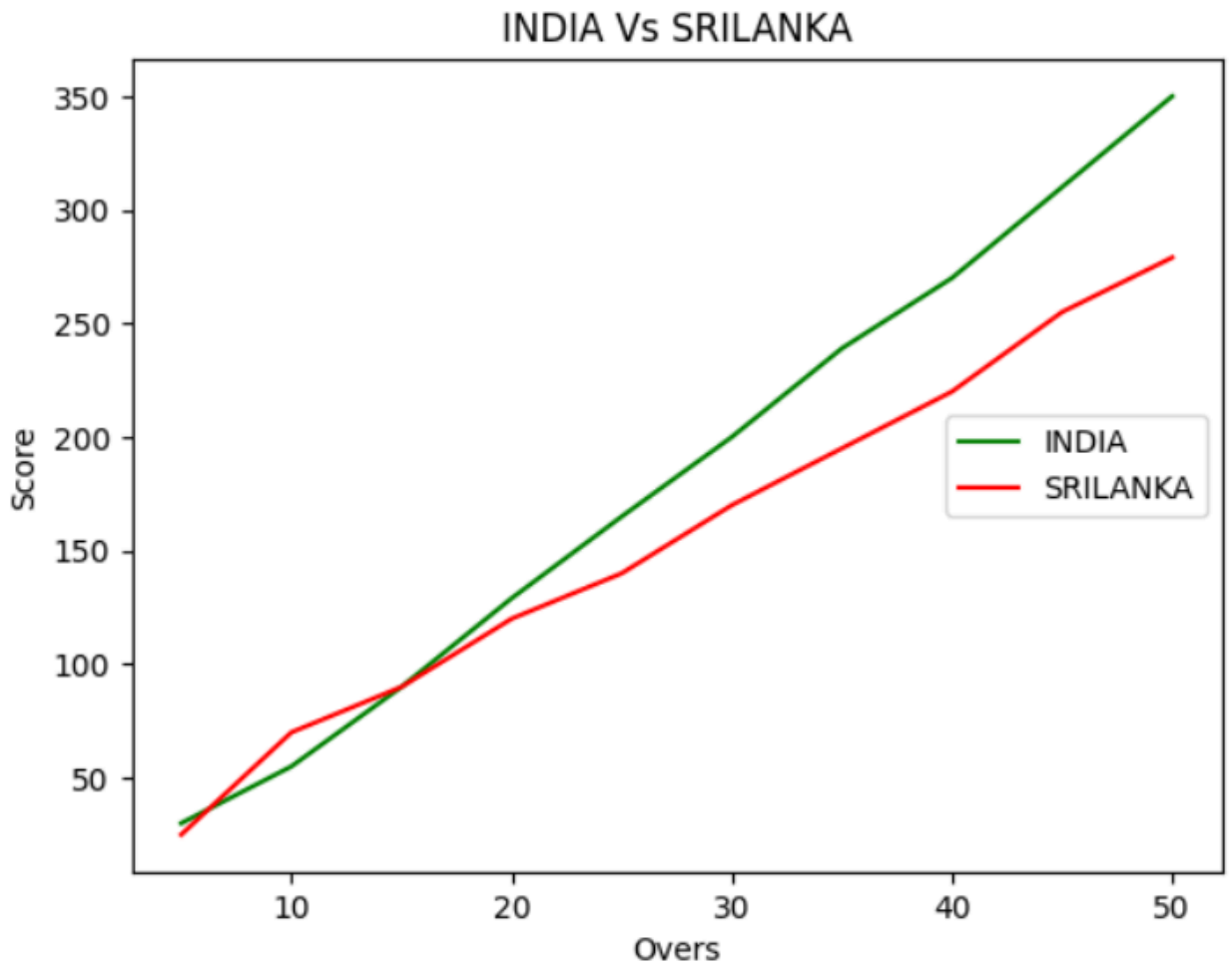
Aim:

To conduct an experiment to show data visualization using line plot

Code:

```
import matplotlib.pyplot as hscmark
import numpy as np
Names = ['SHREE', 'DEV', 'KEERTHI', 'PRIYA', 'SHAN', 'KUMARAN']
xaxis = np.arange(len(Names))
Percentage_hsc = [96, 91, 94, 75, 45, 81]
hscmark.bar(Names, Percentage_hsc)
hscmark.xticks(xaxis, Names, rotation=45)
hscmark.xlabel("Names of Pupil")
hscmark.ylabel("Percentage")
hscmark.title("Comparison of HSC Percentage", fontsize=20, color="green")
hscmark.show()
```

Output:



Result:

Thus,the given data was analyzed successfully and required output was obtained.

Exp No:3.b Conduct an experiment to show data visualization using bar chart

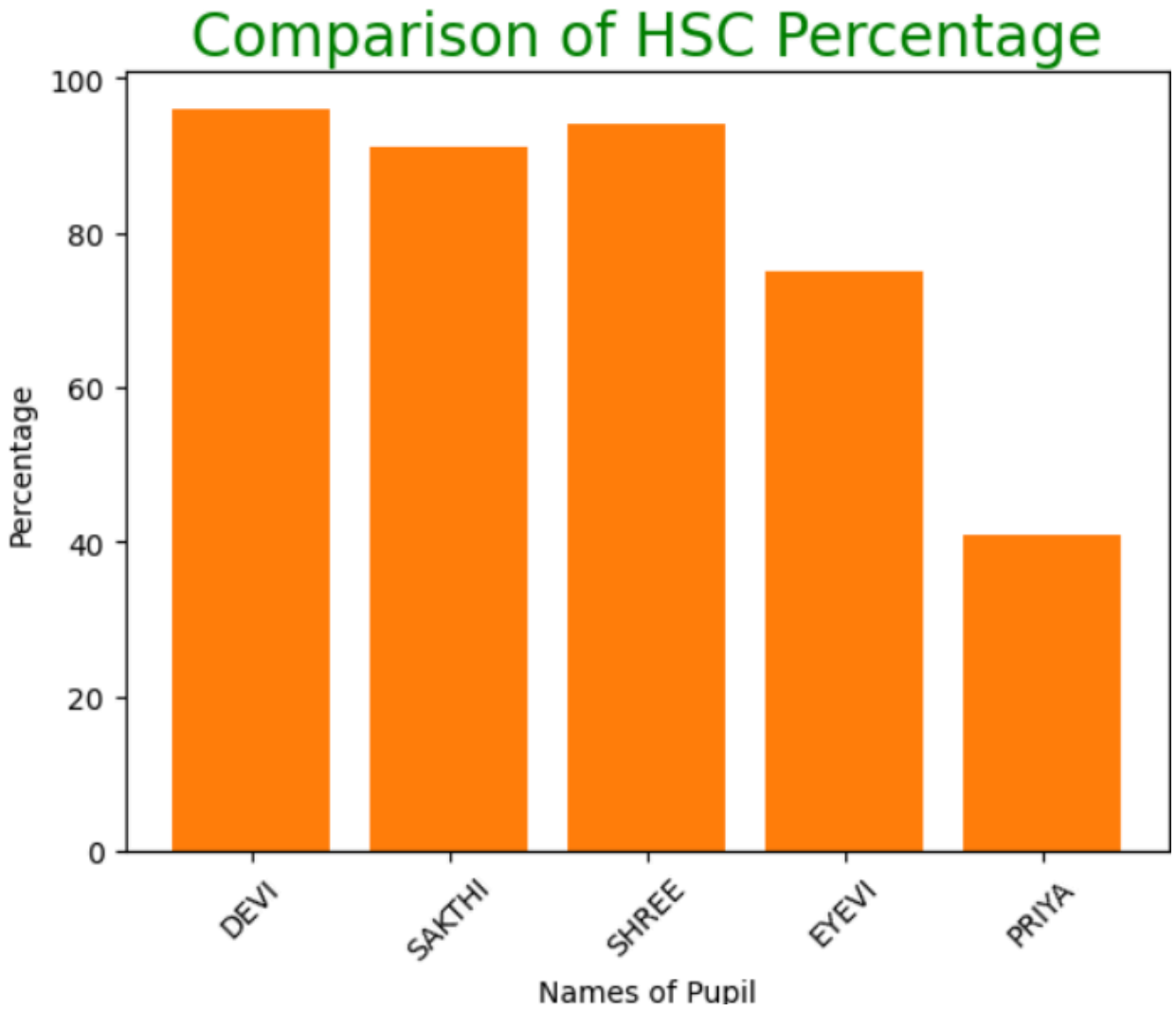
Aim:

To conduct an experiment to show data visualization using bar chart

Code:

```
import matplotlib.pyplot as hscmark
import numpy as np
Names = ['SHREE', 'DEV', 'KEERTHI', 'PRIYA', 'SHAN', 'KUMARAN']
xaxis = np.arange(len(Names))
Percentage_hsc = [96, 91, 94, 75, 45, 81]
hscmark.bar(Names, Percentage_hsc)
hscmark.xticks(xaxis, Names, rotation=45)
hscmark.xlabel("Names of Pupil")
hscmark.ylabel("Percentage")
hscmark.title("Comparison of HSC Percentage",
```

Output:



Result:

Thus,the given data was analyzed successfully and required output was obtained.

Exp No:3.c Conduct an experiment to show data visualization using pie chart

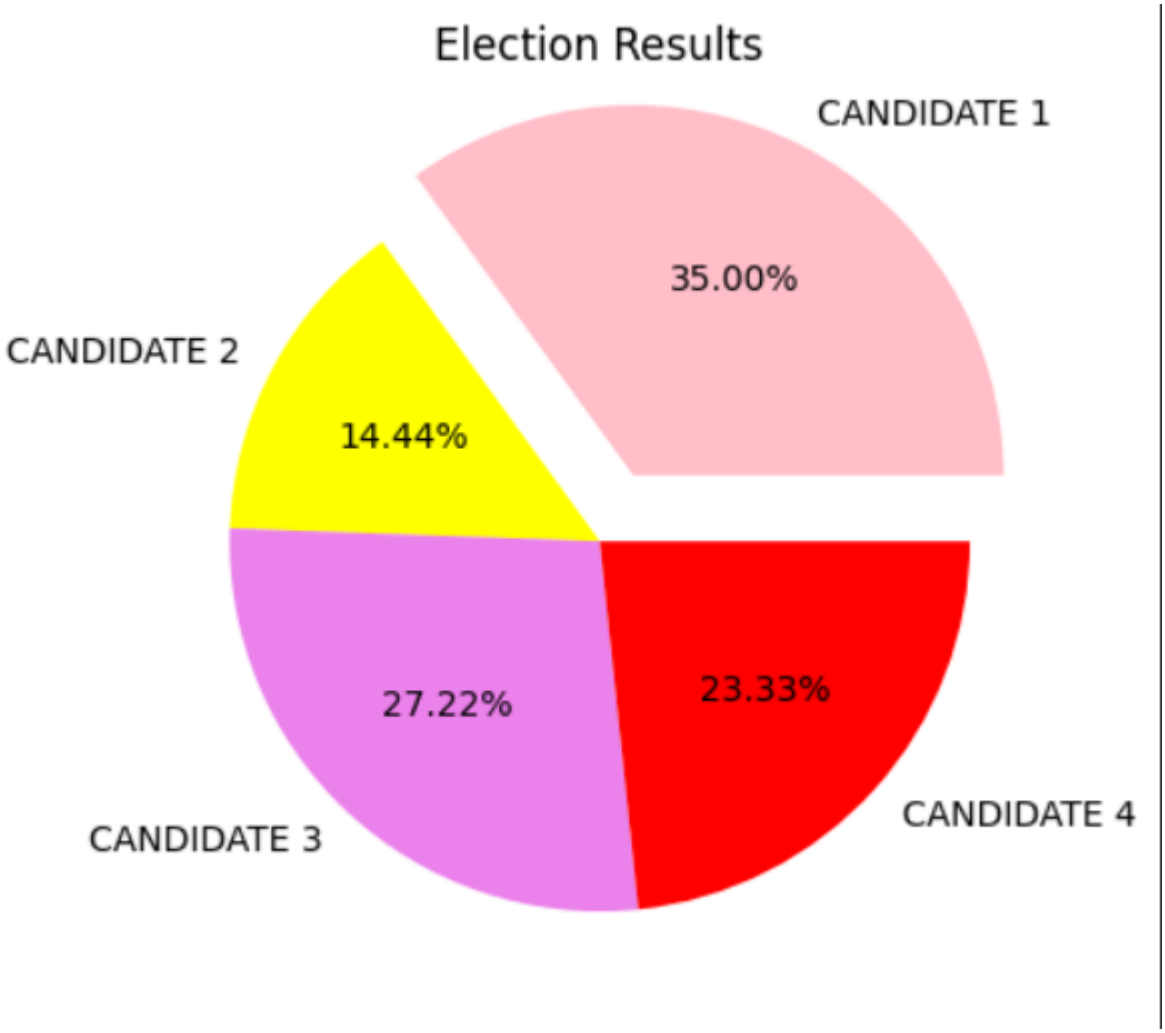
Aim:

To conduct an experiment to show data visualization using pie chart

Code:

```
import matplotlib.pyplot as election
# Election data
labels = ['CANDIDATE 1', 'CANDIDATE 2', 'CANDIDATE 3', 'CANDIDATE 4']
Votes = [315, 130, 245, 210]
colors = ['green', 'yellow', 'red', 'orange']
explode = (0.2, 0, 0, 0)
# Plotting the pie chart
election.pie(Votes, labels=labels, colors=colors, explode=explode, autopct='%0.2f%%')
election.title('Election Results')
election.show()
```

Output:



Result:

Thus,the given data was analyzed successfully and required output was obtained.

Exp No:4 To Count the frequency of occurrence of a word in a body of text is often needed during text processing.

Aim:

To count the frequency of occurrence of a word in a body of text is often needed during text processing.

Code:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg
# Download the gutenberg corpus if not already installed
nltk.download('gutenberg')
nltk.download('punkt') # Ensure the punkt tokenizer models are also downloaded
# Load the text from the gutenberg corpus
sample = gutenberg.raw("austen-emma.txt") # Change to an existing text from the corpus
# Tokenize the sample text
token = word_tokenize(sample)
# Create a list of the first 50 tokens
wlist = []
for i in range(50):
wlist.append(token[i])
# Calculate the frequency of each word in the list
wordfreq = [wlist.count(w) for w in wlist]
# Print the word-frequency pairs
print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

Output:

```
Pairs
[('[', 1), ('Emma', 2), ('by', 1), ('Jane', 1), ('Austen', 1)]
[('1816', 1), (']', 1), ('VOLUME', 1), ('I', 2), ('CHAPTER', 1)]
[('I', 2), ('Emma', 2), ('Woodhouse', 1), (',', 5), ('handsome', 1)]
[(',', 5), ('clever', 1), (',', 5), ('and', 3), ('rich', 1)]
[(',', 5), ('with', 2), ('a', 1), ('comfortable', 1), ('home', 1)]
[('and', 3), ('happy', 1), ('disposition', 1), (',', 5), ('seemed', 1)]
[('to', 1), ('unite', 1), ('some', 1), ('of', 2), ('the', 2)]
[('best', 1), ('blessings', 1), ('of', 2), ('existence', 1), (';', 1)]
[('and', 3), ('had', 1), ('lived', 1), ('nearly', 1), (',', 1)]
[('twenty-one', 1), ('years', 1), ('in', 1), ('the', 2), ('world', 1)]
[('with', 2)]
```

Result:

Thus,the given data was analyzed successfully and required output was obtained.

Exp No:5 Data Collection and Initial Exploration

AIM:

To perform data collection and initial exploration on the given data

PROCEDURE AND CODE:

1. Data collection :

```
import pandas as pd
# Load the dataset
diabetes_df = pd.read_csv(“diabetes.csv”)
# Display the first few rows
print(diabetes_df.head())

from google.colab import files
uploaded = files.upload()

data = pd.read_csv("diabetes.csv")
print(data.head())
```

```
Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
```

2.Initial Exploration :

```
print(data.info())
print(data.describe())
```

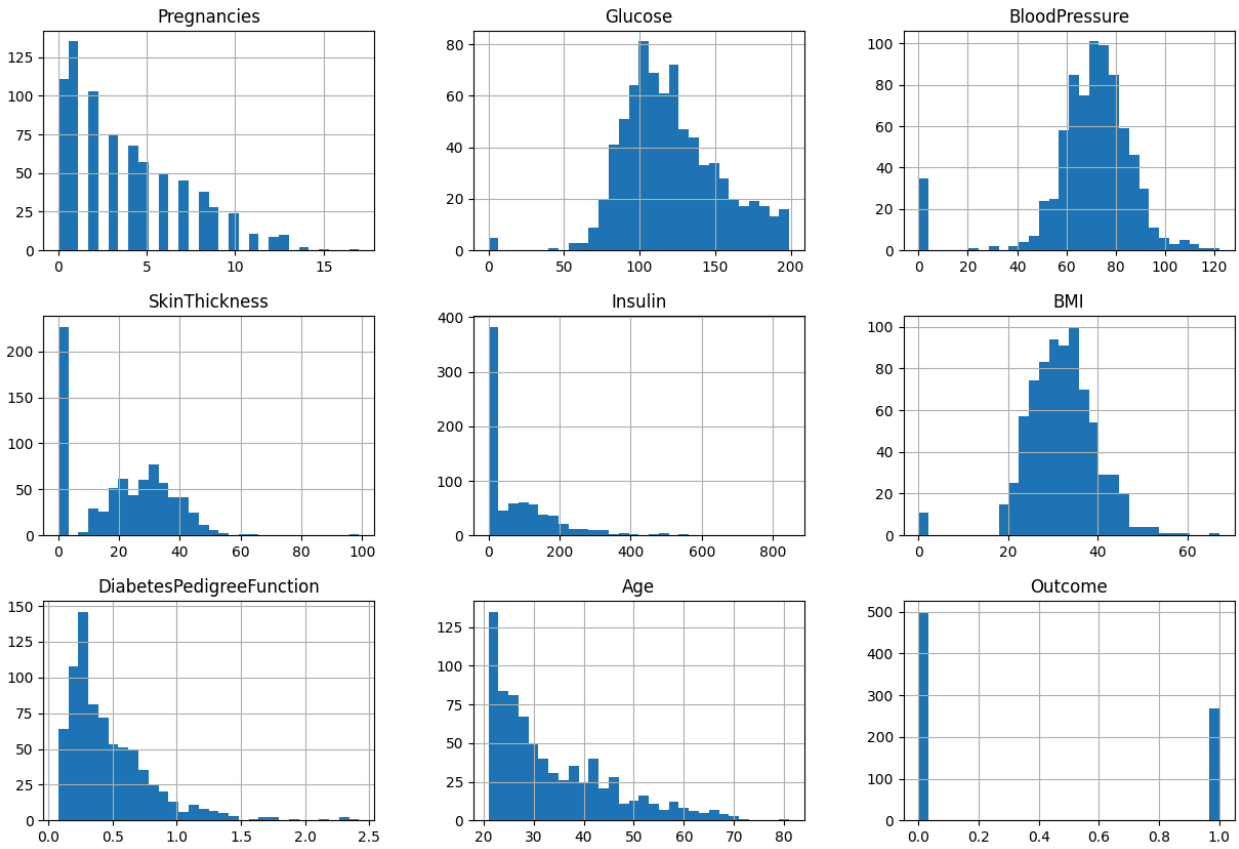
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
```

```
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
datatypes: float64(2), int64(7)
memory usage: 54.1 KB
None
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count   768.000000  768.000000     768.000000     768.000000  768.000000
mean      3.845052  120.894531      69.105469      20.536458   79.799479
std       3.369578   31.972618      19.355807      15.952218  115.244002
min       0.000000    0.000000       0.000000       0.000000    0.000000
25%       1.000000   99.000000      62.000000       0.000000    0.000000
50%       3.000000  117.000000      72.000000      23.000000   30.500000
75%       6.000000  140.250000      80.000000      32.000000  127.250000
max      17.000000  199.000000     122.000000      99.000000  846.000000

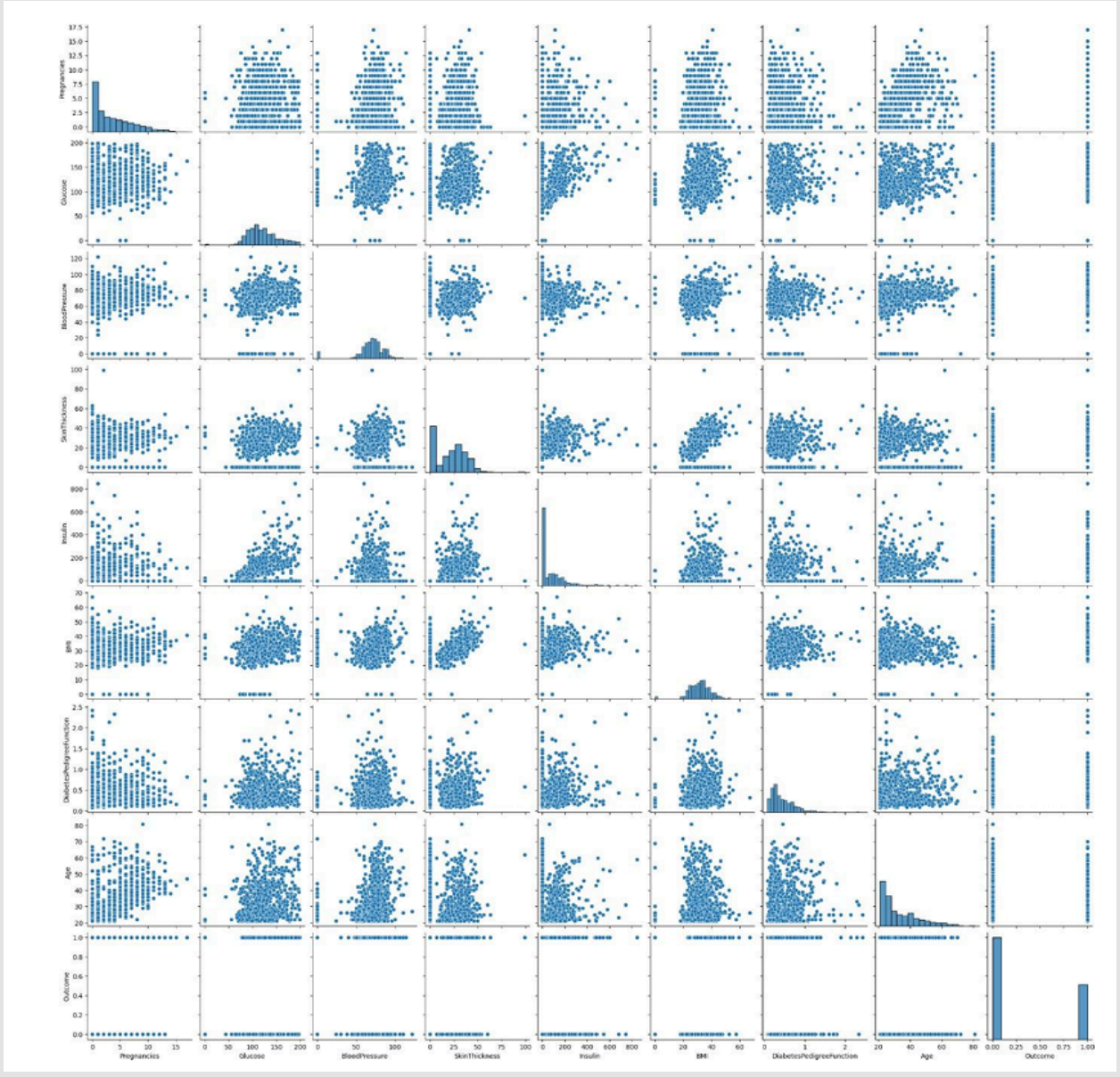
              BMI  DiabetesPedigreeFunction         Age     Outcome
count  768.000000               768.000000  768.000000  768.000000
mean    31.992578                 0.471876   33.240885    0.348958
std      7.884160                 0.331329   11.760232    0.476951
min      0.000000                 0.078000   21.000000    0.000000
25%     27.300000                 0.243750   24.000000    0.000000
50%     32.000000                 0.372500   29.000000    0.000000
75%     36.600000                 0.626250   41.000000    1.000000
max     67.100000                 2.420000   81.000000    1.000000
```

```
data.hist(bins=30, figsize=(15, 10))
plt.show()
```

OUTPUT:




```
sns.pairplot(data)
plt.show()
```



RESULT:

Thus the code was analyzed successfully and the out was verified

Experiment 6: Time Series Analysis of Website Traffic

AIM:

To perform time series analysis of website traffic from the give data

CODE:

```
    import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
from google.colab import files

# Upload Excel file
uploaded = files.upload()

# Get uploaded file name automatically
filename = list(uploaded.keys())[0]

# Load Excel dataset
data = pd.read_excel(filename)

# Convert Date column to datetime
data['Date'] = pd.to_datetime(data['Date'])

# Set Date as index
data = data.set_index('Date')

print("File uploaded successfully!")
print(data.head())

# Time Series Decomposition
decomposition = seasonal_decompose(
   data['Page_Views'],
   model='additive',
   period=7
)

decomposition.plot()
plt.show()

# Moving Average
data['Page_Views_MA'] = data['Page_Views'].rolling(window=30).mean()

data[['Page_Views', 'Page_Views_MA']].plot()
plt.title('Page Views with Moving Average')
```

```
plt.show()

# Anomaly Detection
data['Page_Views_Z'] = (
    (data['Page_Views'] - data['Page_Views'].mean())
    / data['Page_Views'].std()
)

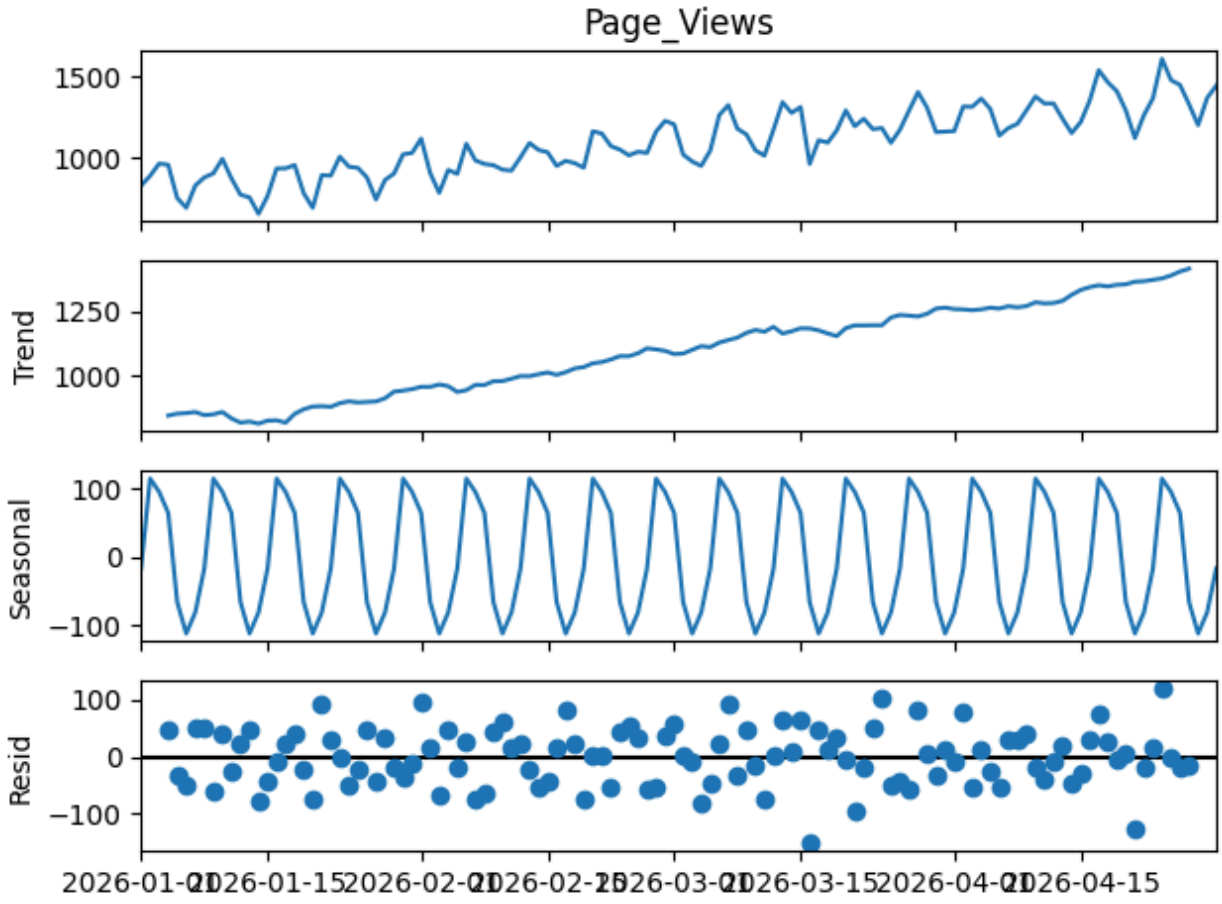
anomalies = data[data['Page_Views_Z'].abs() > 3]

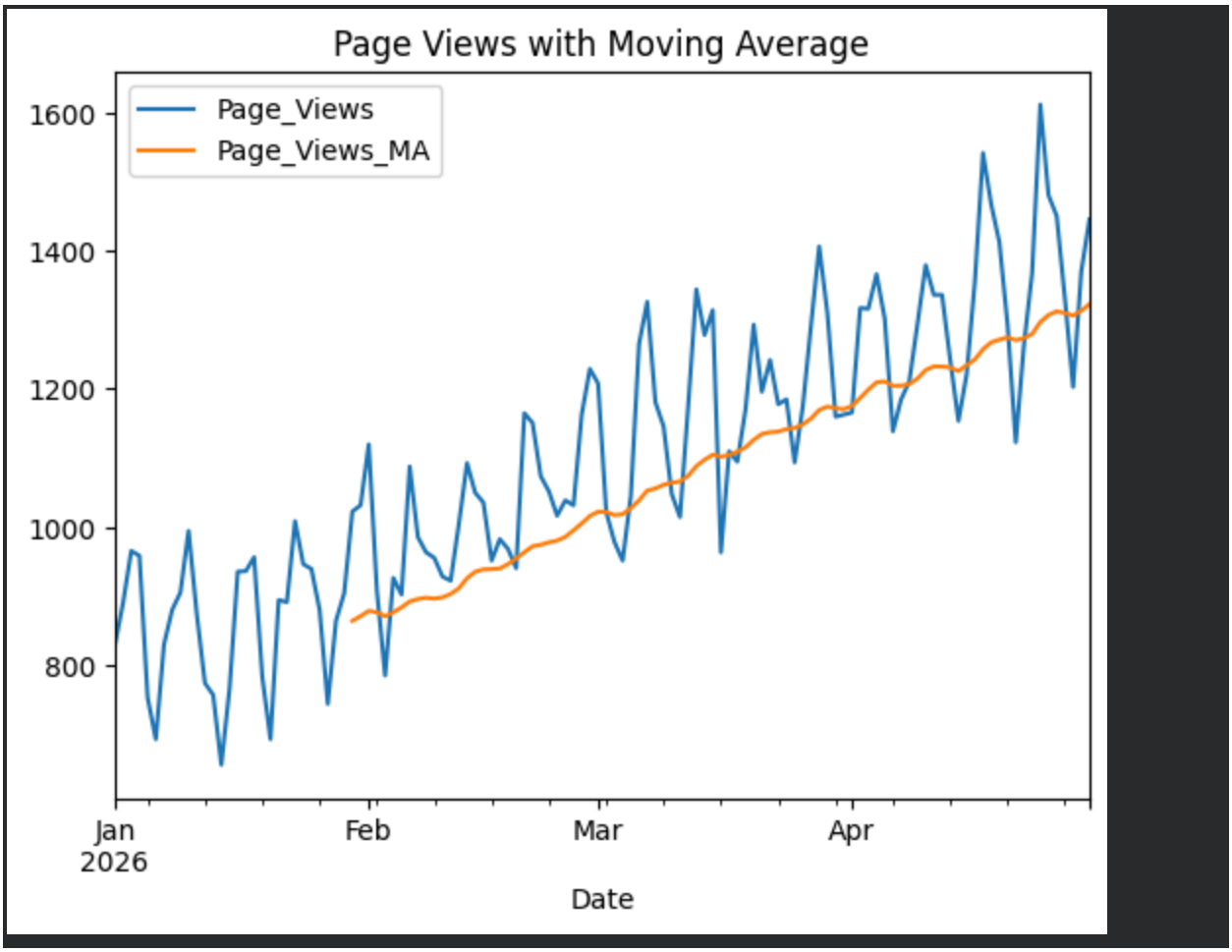
plt.plot(data.index, data['Page_Views'], label='Page Views')

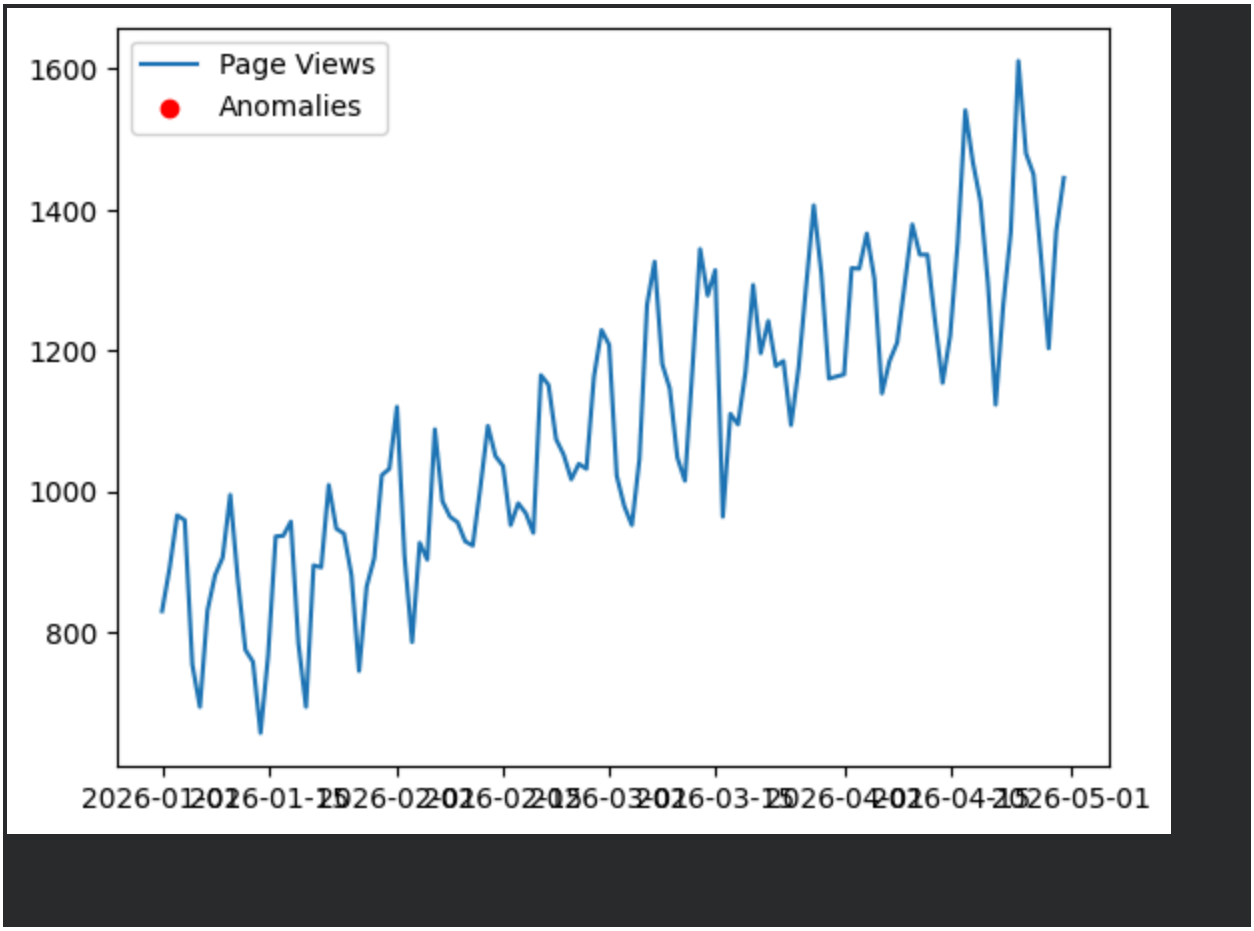
plt.scatter(
    anomalies.index,
    anomalies['Page_Views'],
    color='red',
    label='Anomalies'
)

plt.legend()
plt.show()
```

OUTPUT:


Page_Views
1500
1000
Trend
1250
1000
Seasonal
100
0
−100
Resid
100
0
−100
2026-01-01
2026-01-15
2026-02-01
2026-02-15
2026-03-01
2026-03-15
2026-04-01
2026-04-15


Page Views with Moving Average
Page_Views
Page_Views_MA
1600
1400
1200
1000
800
Jan
2026
Feb
Mar
Apr
Date



RESULT:

Thus,the output was analyzed successfully and the output was verified

Experiment 7:. Random Sampling and Sampling Distribution

AIM:

To perform Random Sampling and Sampling Distribution on the given data.

CODE:

```
    import matplotlib.pyplot as plt
import numpy as np

# Step 1: Generate a population (e.g., normal distribution)
population_mean = 50
population_std = 10
population_size = 100000
population = np.random.normal(population_mean, population_std, population_size)

# Step 2: Random sampling
sample_sizes = [30, 50, 100]  # different sample sizes to consider
num_samples = 1000  # number of samples for each sample size

sample_means = {}

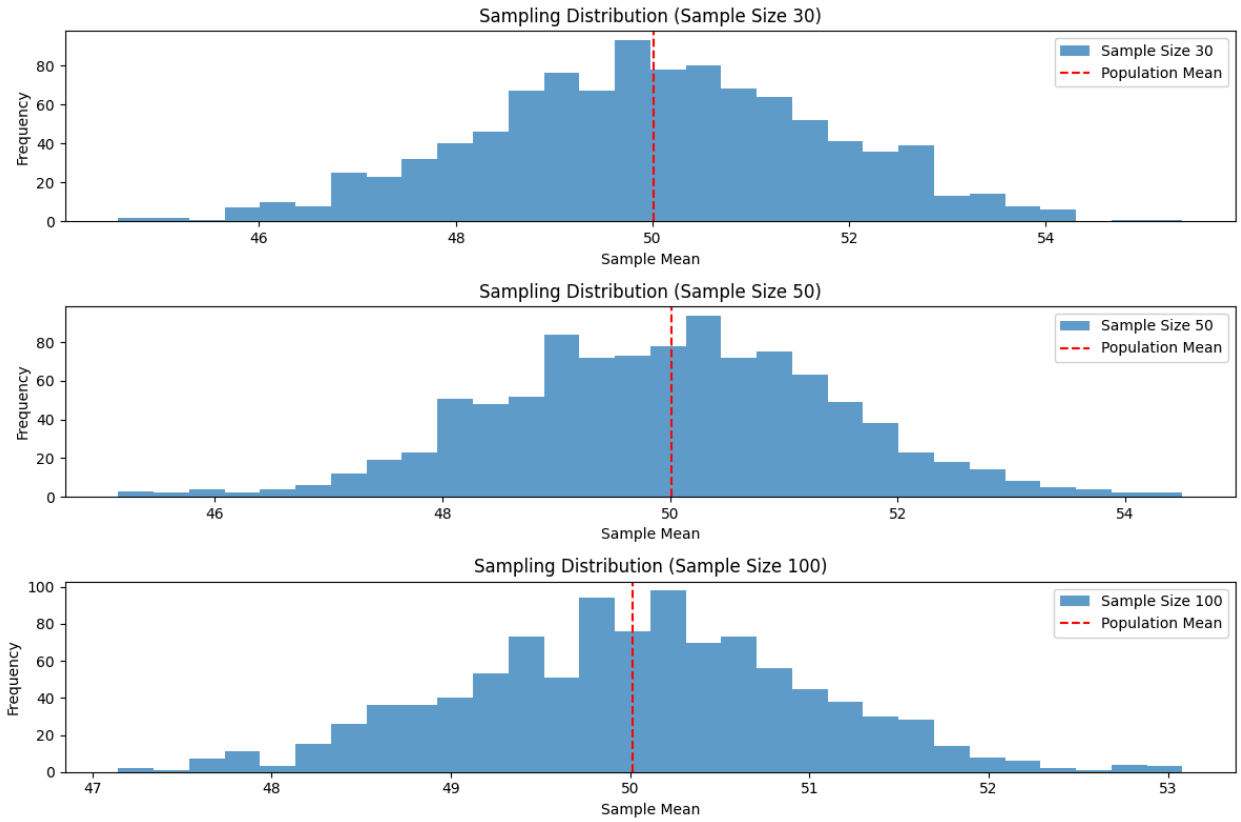
for size in sample_sizes:
    sample_means[size] = []
    for _ in range(num_samples):
        sample = np.random.choice(population, size=size, replace=False)
        sample_means[size].append(np.mean(sample))

# Step 3: Plotting sampling distributions
plt.figure(figsize=(12, 8))

for i, size in enumerate(sample_sizes):
    plt.subplot(len(sample_sizes), 1, i + 1)
    plt.hist(sample_means[size], bins=30, alpha=0.7, label=f"Sample Size {size}")
    plt.axvline(
        np.mean(population),
        color="red",
        linestyle="dashed",
        linewidth=1.5,
        label="Population Mean",
    )
    plt.title(f"Sampling Distribution (Sample Size {size})")
    plt.xlabel("Sample Mean")
    plt.ylabel("Frequency")
    plt.legend()
```

```
plt.tight_layout()
plt.show()
```

OUTPUT:



RESULT:

Thus, the code was analyzed successfully and the output was verified.

Experiment 8: Conduct Z test for the Data Given using Python Objective

AIM:

To Conduct z test for the data given using python objective.

CODE:

```
import numpy as np
import scipy.stats as stats

# Step 1: Input sample data as a single string instead of a multiline array
data_string = "152 148 151 149 147 153 150 148 152 149 151 150 149 152 151 148 150 152
149 150 148 153 151 150 149 152 148 151 150 153"
sample_data = np.fromstring(data_string, sep=" ")

# Step 2: Population mean under Null Hypothesis (H0: mu = 150)
population_mean = 150

# Step 3: Calculate sample statistics
sample_mean = np.mean(sample_data)
sample_std = np.std(sample_data, ddof=1)
n = len(sample_data)

# Step 4: Calculate T-statistic and P-value
t_statistic, p_value = stats.ttest_1samp(sample_data, population_mean)

# Step 5: Print results
print(f"Sample Mean: {sample_mean:.2f}")
print(f"Sample Std Dev: {sample_std:.4f}")
print(f"Degrees of Freedom: {n - 1}")
print(f"T-Statistic: {t_statistic:.4f}")
print(f"P-Value: {p_value:.4f}")

# Step 6: Decision rule
alpha = 0.05
if p_value < alpha:
    print("Decision: Reject the null hypothesis (Significant difference).")
else:
    print(
        "Decision: Fail to reject the null hypothesis (No significant difference)."
```

OUTPUT:

Sample Std Dev: 1.7100
Degrees of Freedom: 29
T-Statistic: 0.6406
P-Value: 0.5268
Decision: Fail to reject the null hypothesis (No significant difference)

RESULT:

Thus, the code was analyzed successfully and the output was verified.

Exp No:9 Experiment to understand Matplotlib library use cases in Data Science through visualization

AIM:

To understand matplotlib library use cases in data science through visualization.

CODE:

```
import pandas as pd
import numpy as np
import seaborne as sns
import matplotlib.pyplot as plt

%matplotlib inline

data = pd.read_csv('/content/Iris_Dataset.csv')
data

data.info()

data.describe()

data.value_counts('variety')

sns.countplot(x='variety', data=data)
plt.show()

dummies = pd.get_dummies(data['variety'])

FinalDataset = pd.concat(
   [pd.get_dummies(data['variety']), data.iloc[:, [0, 1, 2, 3]]],
   axis=1
)

FinalDataset.head()

sns.scatterplot(
   x='SepalLengthCm',
   y='SepalWidthCm',
   hue='variety',
   data=data
)
plt.show()

sns.scatterplot(
```

```
    x='PetalLengthCm',
    y='PetalWidthCm',
    hue='variety',
    data=data
)
plt.show()

sns.pairplot(data, hue='variety', height=3)
plt.show()

sns.FacetGrid(
    data,
    hue='variety',
    height=5
).map(sns.histplot, 'PetalLengthCm').add_legend()
plt.show()

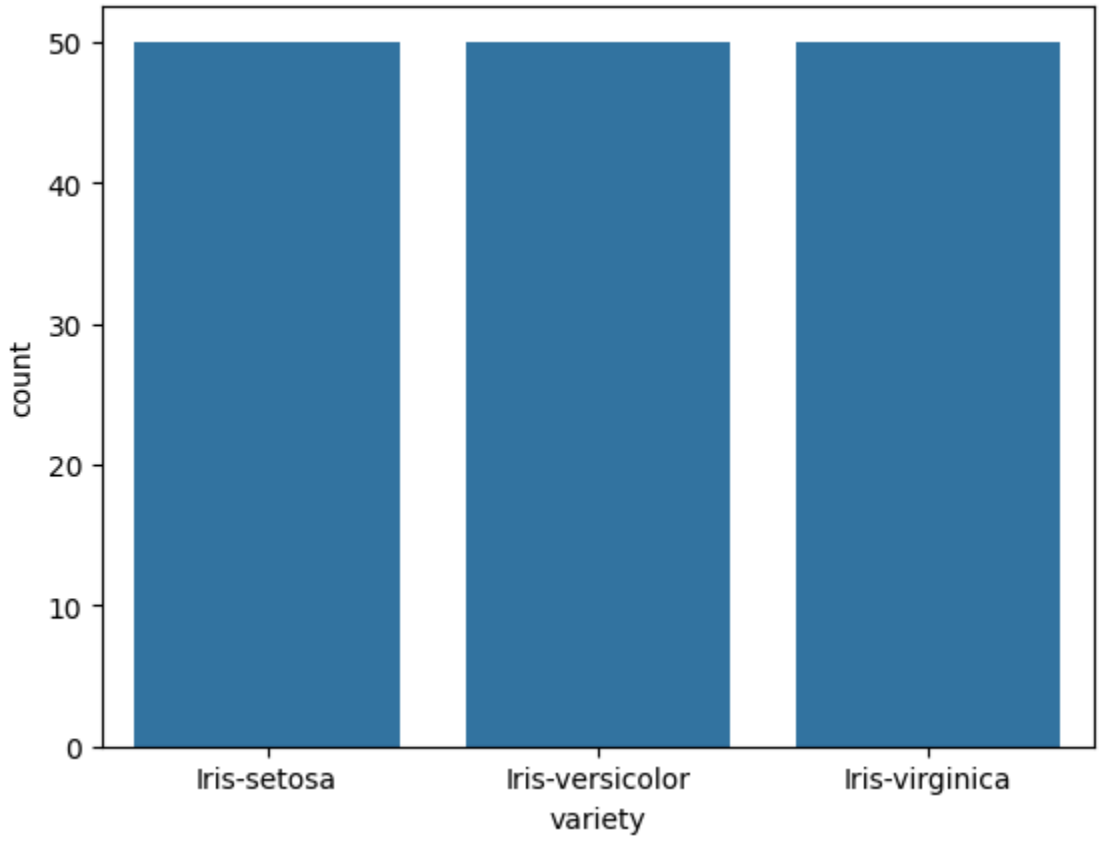
sns.FacetGrid(
    data,
    hue='variety',
    height=5
).map(sns.histplot, 'PetalWidthCm').add_legend()
plt.show()

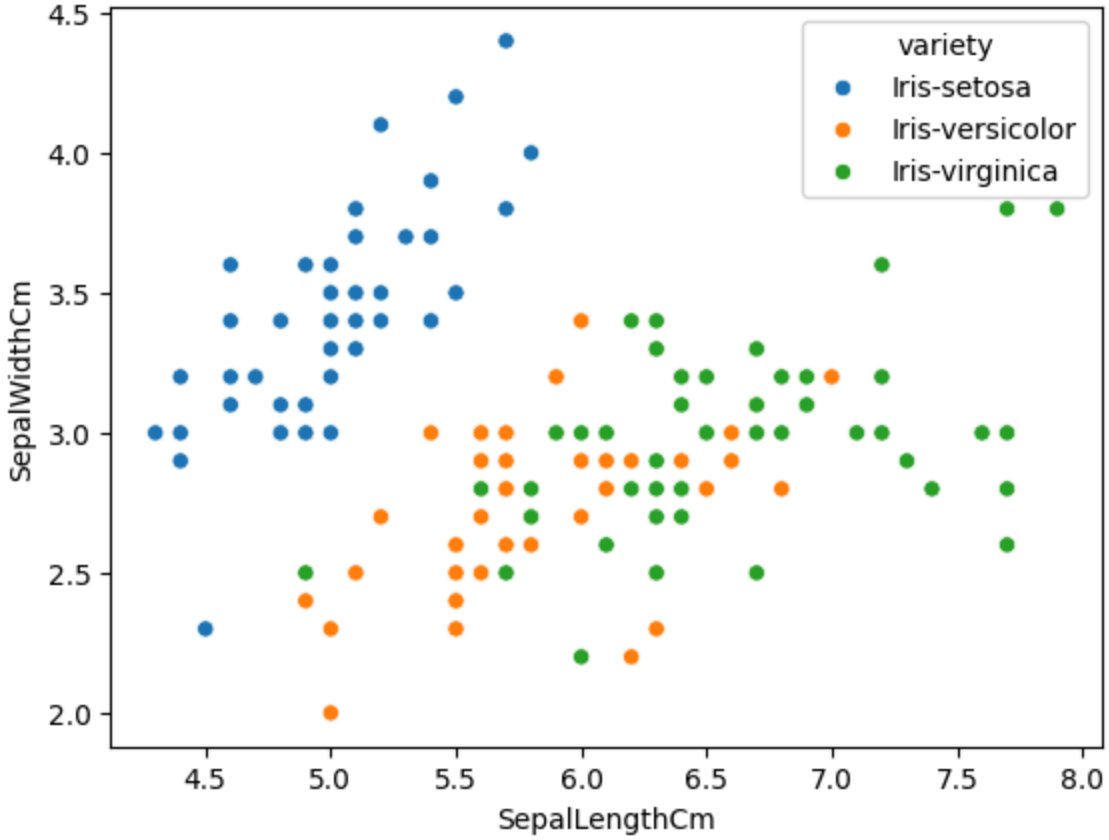
sns.FacetGrid(
    data,
    hue='variety',
    height=5
).map(sns.histplot, 'SepalLengthCm').add_legend()
plt.show()

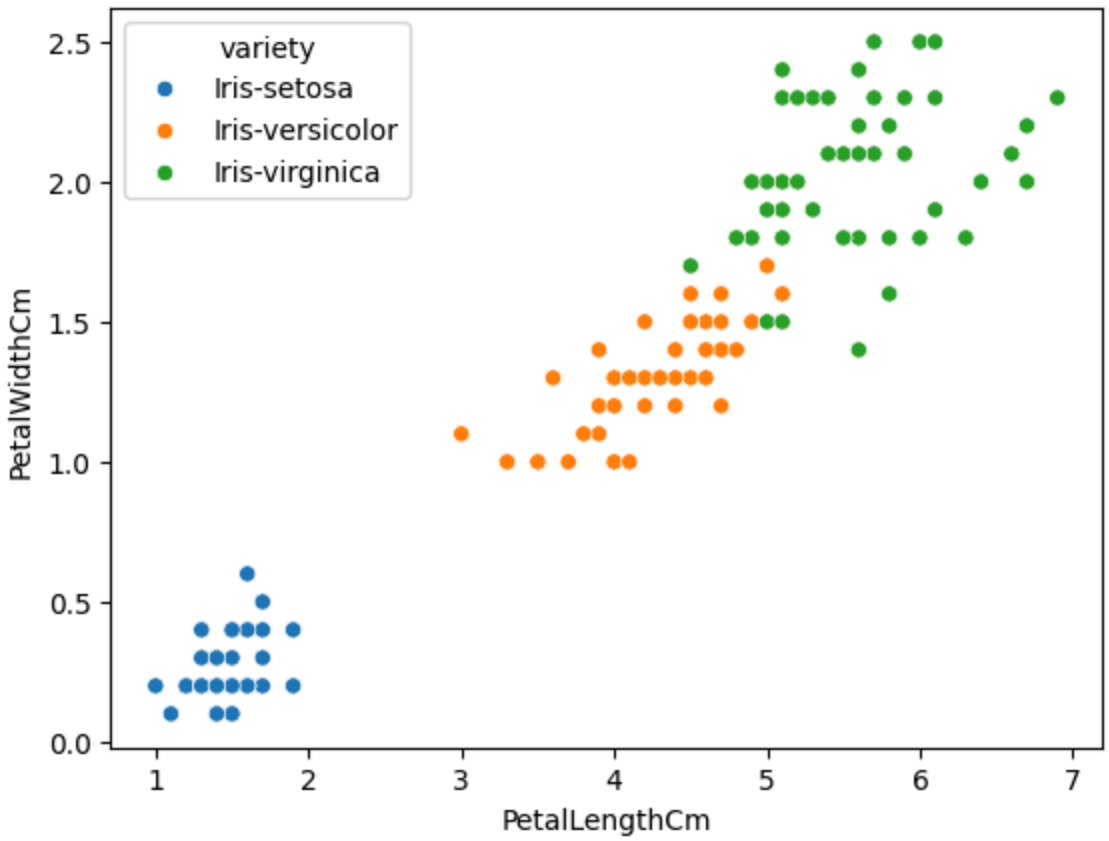
sns.FacetGrid(
    data,
    hue='variety',
    height=5
).map(sns.histplot, 'SepalWidthCm').add_legend()
plt.show()

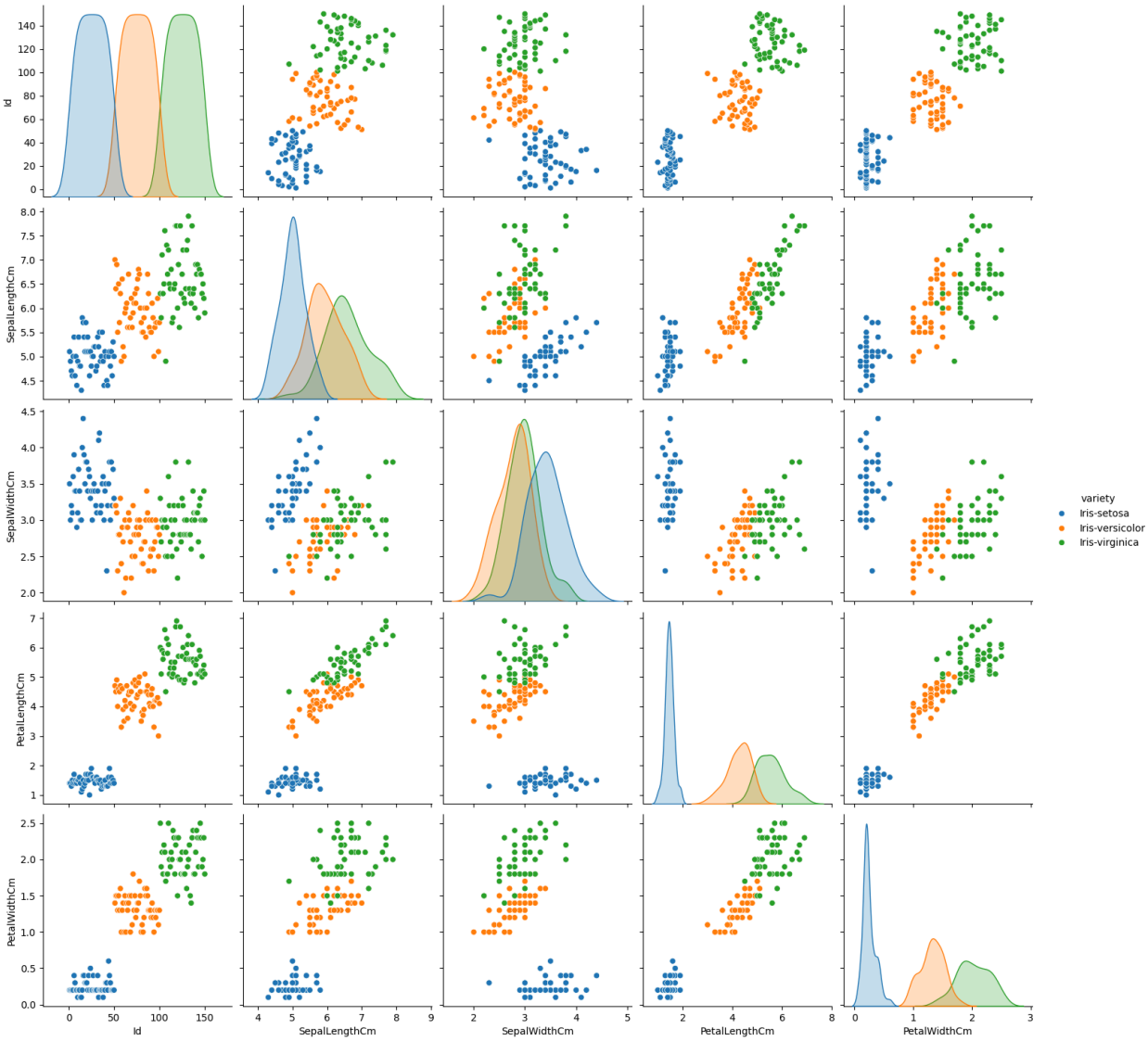
OUTPUT:
            <class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
```

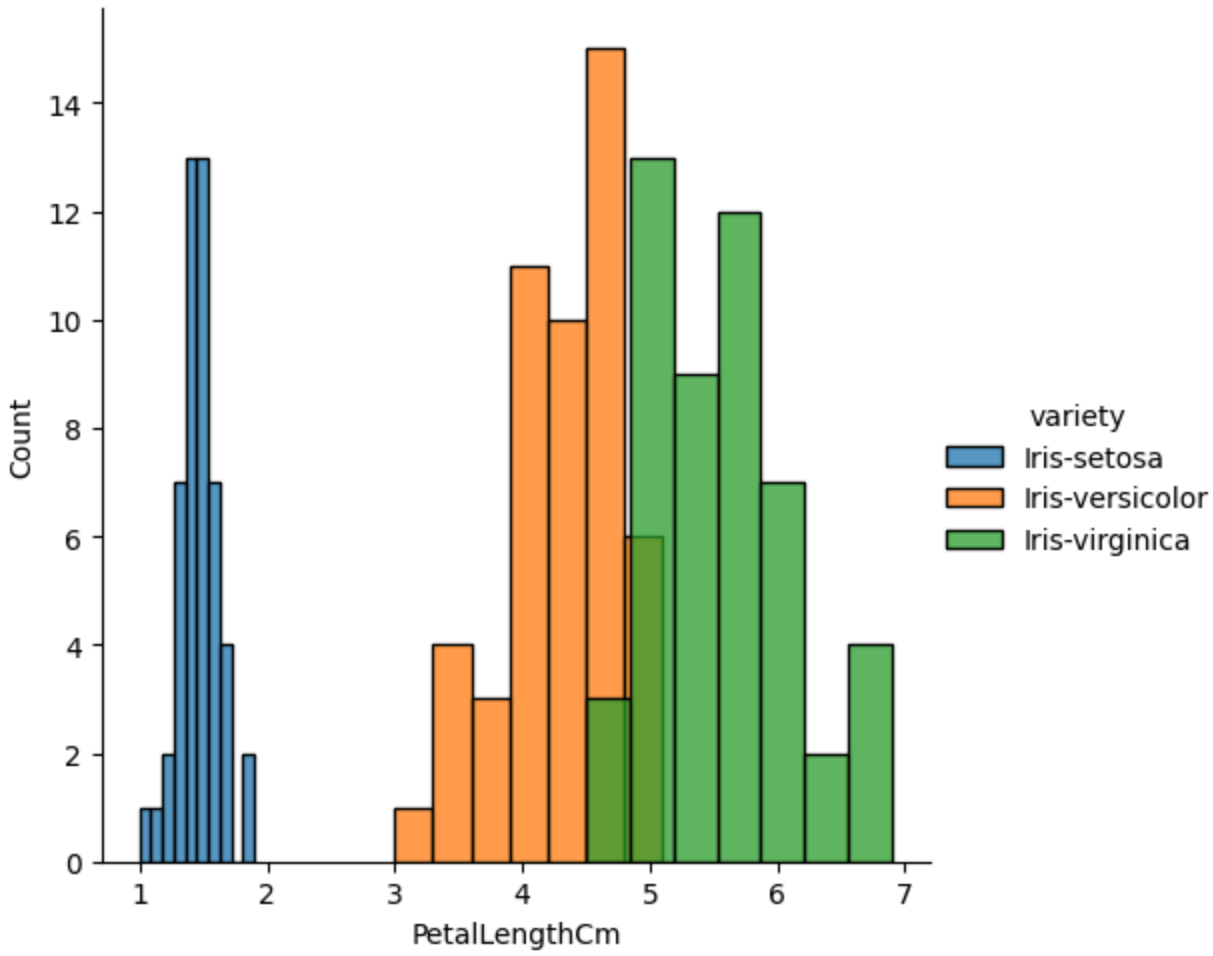
```
 0           id              150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   variety        150 non-null    object
datatypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

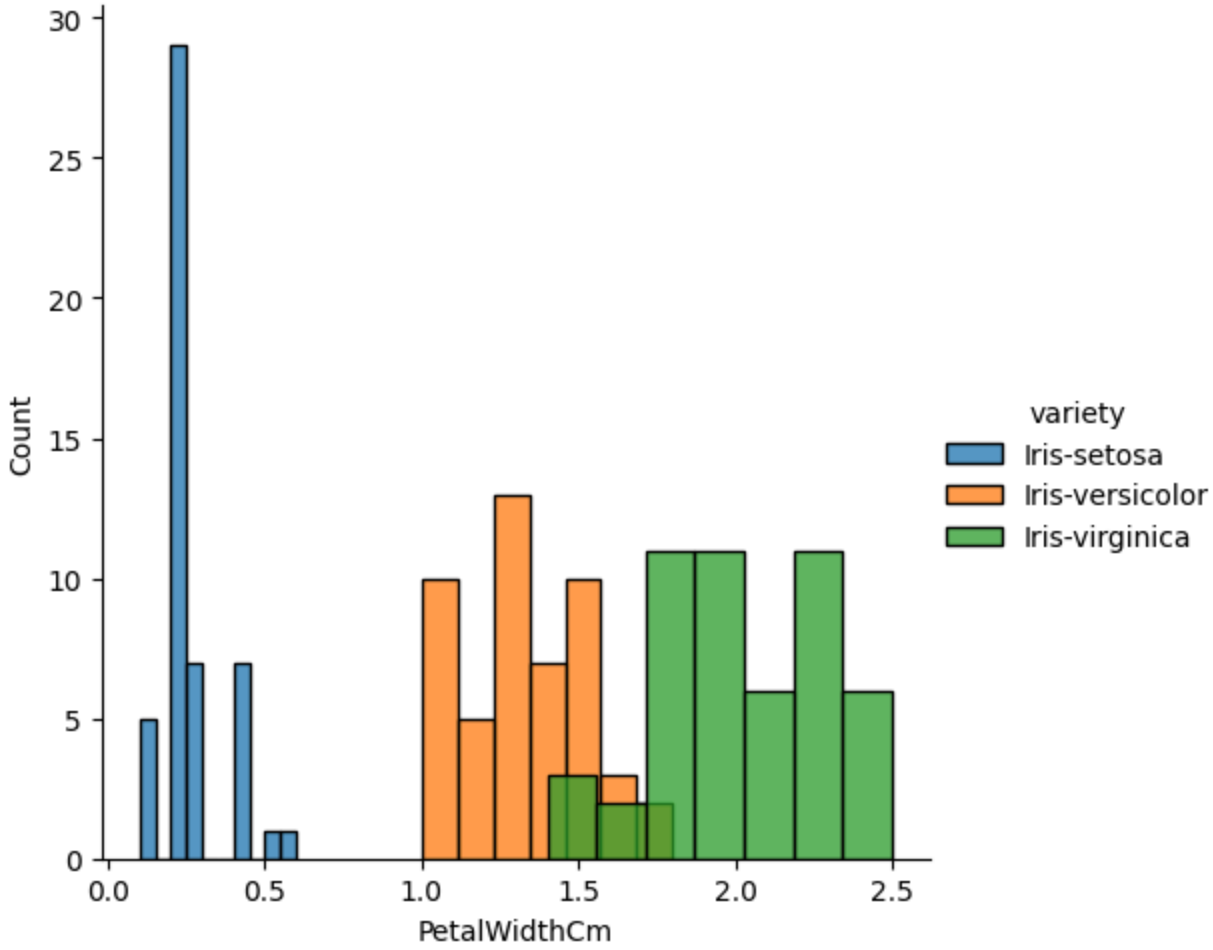


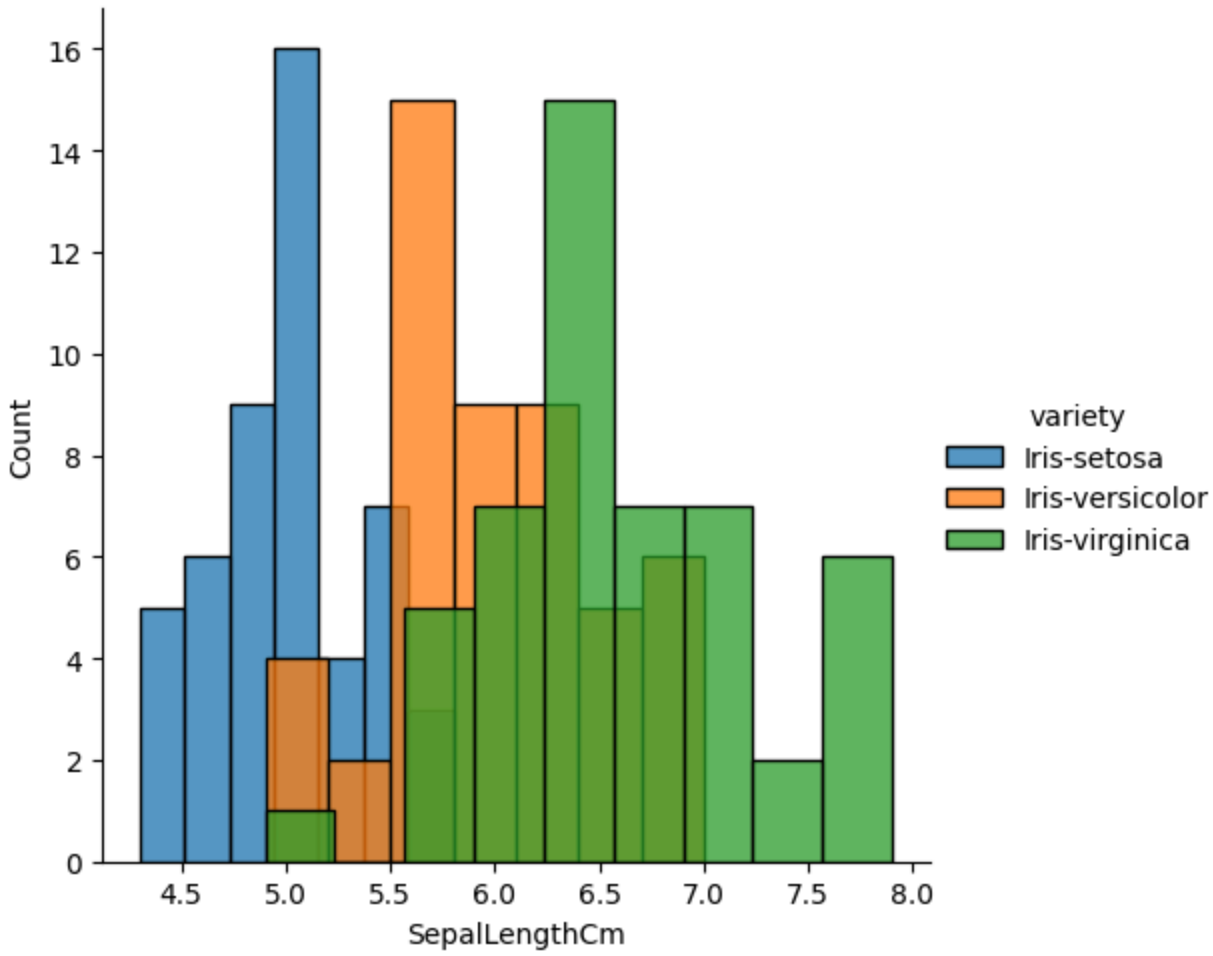

variety
Iris-setosa
Iris-versicolor
Iris-virginica
SepalWidthCm
SepalLengthCm
4.5
4.0
3.5
3.0
2.5
2.0
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0

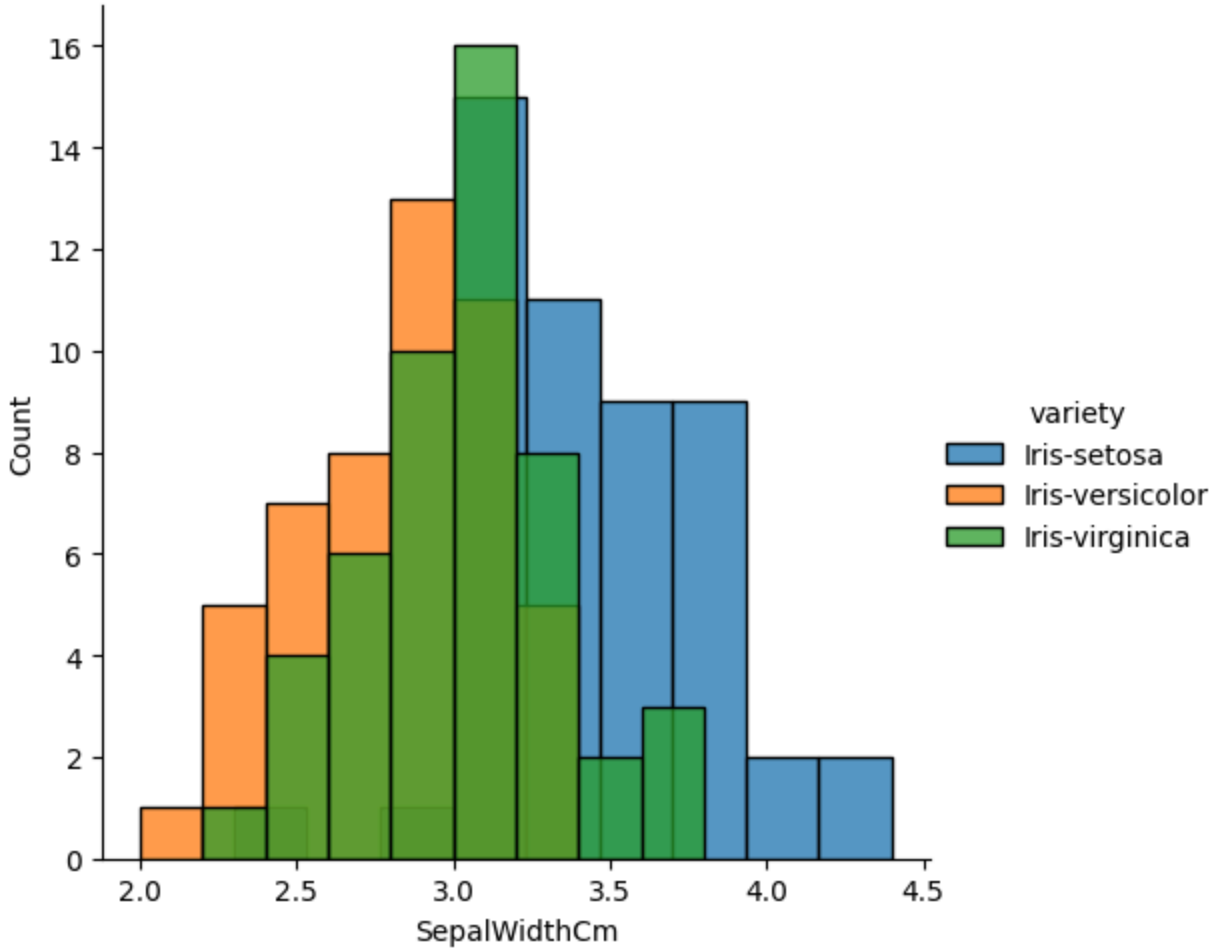

variety
Iris-setosa
Iris-versicolor
Iris-virginica
PetalWidthCm
PetalLengthCm
0.0
0.5
1.0
1.5
2.0
2.5
1
2
3
4
5
6
7


Id
SepalLengthCm
SepalWidthCm
PetalLengthCm
PetalWidthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica


Count
PetalLengthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica


30
25
20
15
10
5
0
Count
0.0
0.5
1.0
1.5
2.0
2.5
PetalWidthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica


Count
SepalLengthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica



RESULT :

Thus the code was analyzed successfully and the out was verified

Exp No:10 Experiment to understand array function in Data science.

AIM:

To understand array function in data science

CODE:

```
import numpy as np

array=np.random.randint(1,100,9)
array

np.sqrt(array)

array.ndim

new_array=array.reshape(3,3)

new_array

new_array.ndim

new_array.ravel()

newm=new_array.reshape(3,3)

newm

newm[2,1:3]

newm[1:2,1:3]

new_array[0:3,0:0]

new_array[0:2,0:1]

new_array[0:3,0:1]

New_array[1:3]
```

OUTPUT:

```
array([[38, 56, 62],
       [ 8, 67, 93]])
```

RESULT:

Thus,the given data was analyzed successfully and required output was obtained.

Exp No:11 Experiment to understand pandas library use cases in Data science

AIM:

To understand pandas library use cases in data science

CODE:

```
from google.colab import files
uploaded = files.upload()
import numpy as np
import pandas as pd

# Create DataFrame
list=[[1,'Smith',50000],[2,'Jones',60000]]

df=pd.DataFrame(list)
df

# Assign column names
df.columns=['Empd','Name','Salary']
df

# Display information
df.info()

# Read employee.csv
df=pd.read_csv("/content/employee.csv")

df.head()

# Display last records
df.tail()

# Display information
df.info()

# Display salary column
df.salary

# Check type
type(df.salary)

# Mean
df.salary.mean()
```

```
# Median
df.salary.median()

# Mode
df.salary.mode()

# Variance
df.salary.var()

# Standard deviation
df.salary.std()

# Descriptive statistics
df.describe()

# Descriptive statistics for all columns
df.describe(include='all')=

# Display column names
empCol=df.columns
empCol

# Display values
emparray=df.values
emparray

# Create Employee DataFrame
employee_DF=pd.DataFrame(emparray,columns=empCol)
employee_DF
```

OUTPUT:

```
        Upload widget is only available when the cell has been executed in the current browser
session. Please rerun this cell to enable.
Saving employee.csv to employee.csv
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Empd    2 non-null      int64
 1   Name    2 non-null      object
 2   Salary  2 non-null      int64
dtypes: int64(2), object(1)
```

memory usage: 180.0+ bytes
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 3 columns):
 #   Column Non-Null Count Dtype
--- ------ -------------- -----
 0   emp id 7 non-null   int64
 1   name   7 non-null   object
 2   salary 7 non-null   int64
dtypes: int64(2), object(1)
memory usage: 300.0+ bytes

	emp id	name	salary
0	1	SREE VARSSINI K S	5000
1	2	SREEMATHI B	6000
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000
4	5	SRI AKASH U G	8000
5	6	SRI HARSHAVARDHANAN R	3000
6	7	SRI HARSHAVARDHANAN R	6000

RESULT:

Thus,the given data was analyzed successfully and required output was obtained.

Exp No:12 Experiment to detect outliers in a given data set.

AIM:

To detect outliers in a given data set.

CODE:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Load CSV file
df = pd.read_csv("experiment12_outliers.csv")

# Convert column to NumPy array
array = df["Value"].values

print("Array:")
print(array)

# Mean
print("\nMean:")
print(array.mean())

# Percentiles
print("\n25th Percentile (Q1):")
print(np.percentile(array, 25))

print("\n50th Percentile (Median):")
print(np.percentile(array, 50))

print("\n75th Percentile (Q3):")
print(np.percentile(array, 75))

print("\n100th Percentile:")
print(np.percentile(array, 100))

# Outlier detection using IQR
def outDetection(array):
    Q1, Q3 = np.percentile(array, [25, 75])
    IQR = Q3 - Q1
    lr = Q1 - (1.5 * IQR)
```

```
    ur = Q3 + (1.5 * IQR)
    return lr, ur

lr, ur = outDetection(array)

print("\nLower Range:", lr)
print("Upper Range:", ur)

# Detect outliers
outliers = array[(array < lr) | (array > ur)]

print("\nOutliers:")
print(outliers)

# Remove outliers
new_array = array[(array > lr) & (array < ur)]

print("\nArray after removing outliers:")
print(new_array)

# Distribution plot
sns.displot(array)
plt.title("Distribution of Data")
plt.show()
```

OUTPUT:

```
            Upload widget is only available when the cell has been executed in the current browser
session. Please rerun this cell to enable.
Saving experiment12_outliers.csv to experiment12_outliers.csv
Array:
[27 50 44  6 58 61 23 86 67 20 75  7 79 61 90 54]

Mean:
50.5

25th Percentile (Q1):
26.0

50th Percentile (Median):
56.0

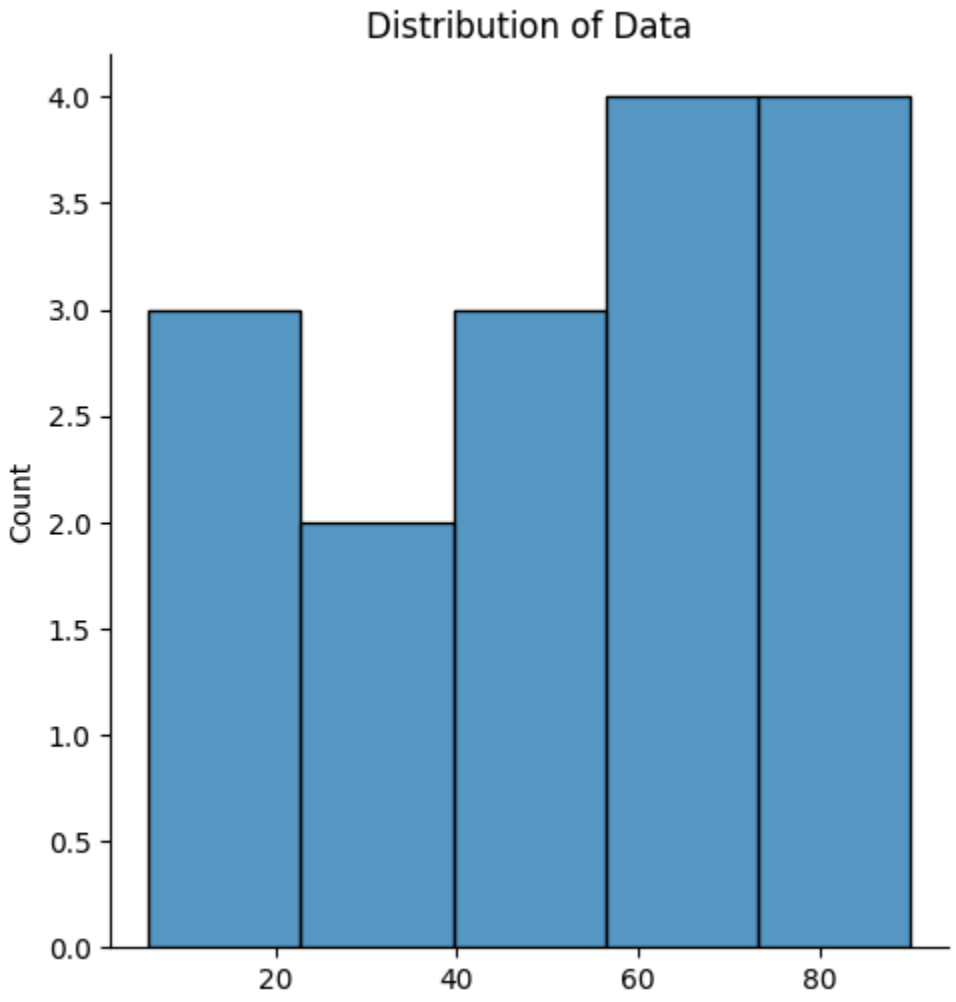
75th Percentile (Q3):
69.0
```

100th Percentile:
90.0

Lower Range: -38.5
Upper Range: 133.5

Outliers:
[]

Array after removing outliers:
[27 50 44 6 58 61 23 86 67 20 75 7 79 61 90 54]



RESULT:

Thus,the given data was analyzed successfully and required output was obtained.

Exp No:13 Experiment to handle missing values and inappropriate data in a given data set

AIM:

To understand and handle missing values and inappropriate data in a given data set.

CODE:

```
from google.colab import files
files.upload()
import numpy as np
import pandas as pd

df = pd.read_csv("Hotel_Dataset.csv")
df

# Identify duplicate row
df.duplicated()

# Display information
df.info()

# Remove duplicate rows
df.drop_duplicates(inplace=True)
df

# Length of dataframe
len(df)

# Reset index
index = np.array(list(range(0, len(df))))
df.set_index(index, inplace=True)
df

# Remove duplicate Age_Group column
df.drop(['Age_Group.1'], axis=1, inplace=True)
df

# Replace inappropriate values with NaN
df.CustomerID.loc[df.CustomerID < 0] = np.nan
df.Bill.loc[df.Bill < 0] = np.nan
df.EstimatedSalary.loc[df.EstimatedSalary < 0] = np.nan
df['NoOfPax'].loc[(df['NoOfPax'] < 1) | (df['NoOfPax'] > 20)] = np.nan

df
```

```
# Check unique values
df.Age_Group.unique()
df.Hotel.unique()

# Correct inappropriate hotel name
df.Hotel.replace(['Ibys'], 'Ibis', inplace=True)

# Correct food preference values
df.FoodPreference.replace(['Vegetarian', 'veg'], 'Veg', inplace=True)
df.FoodPreference.replace(['non-Veg'], 'Non-Veg', inplace=True)

# Fill missing values
df.EstimatedSalary.fillna(round(df.EstimatedSalary.mean()), inplace=True)
df.NoOfPax.fillna(round(df.NoOfPax.median()), inplace=True)
df['Rating(1-5)'].fillna(round(df['Rating(1-5)'].median()), inplace=True)
df.Bill.fillna(round(df.Bill.mean()), inplace=True)

df
```

OUTPUT:

```
            <class 'pandas.core.frame.DataFrame'>
RangeIndex: 11 entries, 0 to 10
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerID       11 non-null     int64
 1   Age_Group        11 non-null     object
 2   Rating(1-5)      11 non-null     int64
 3   Hotel            11 non-null     object
 4   FoodPreference   11 non-null     object
 5   Bill             10 non-null     float64
 6   NoOfPax          11 non-null     int64
 7   EstimatedSalary  11 non-null     int64
 8   Age_Group.1      11 non-null     object
```

```
df.Bill.fillna(round(df.Bill.mean()), inplace=True)
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary
0	1.0	20-25	4	Ibis	Veg	1300.0	2.0	40000.0
1	2.0	30-35	5	LemonTree	Non-Veg	2000.0	3.0	59000.0
2	3.0	25-30	6	RedFox	Veg	1322.0	2.0	30000.0
3	4.0	20-25	-1	LemonTree	Veg	1234.0	2.0	120000.0
4	5.0	35+	3	Ibis	Veg	989.0	2.0	45000.0
5	6.0	35+	3	Ibis	Non-Veg	1909.0	2.0	122220.0
6	7.0	35+	4	RedFox	Veg	1000.0	2.0	21122.0
7	8.0	20-25	7	LemonTree	Veg	2999.0	2.0	345673.0
8	9.0	25-30	2	Ibis	Non-Veg	3456.0	3.0	96755.0
9	10.0	30-35	5	RedFox	Non-Veg	1801.0	4.0	87777.0

RESULT:

Thus,the given data was analyzed successfully and required output was obtained.

Exp No:14 Experiment to understand feature scaling.

AIM:

To understand the importance of feature scaling.

CODE:

```
from google.colab import files
files.upload()
import numpy as np
import pandas as pd

df = pd.read_csv('/content/pre-process_datasample.csv')
df

df.Country.fillna(df.Country.mode()[0], inplace=True)

features = df.iloc[:, :-1].values
label = df.iloc[:, -1].values

from sklearn.impute import SimpleImputer

age = SimpleImputer(strategy="mean", missing_values=np.nan)
Salary = SimpleImputer(strategy="mean", missing_values=np.nan)

age.fit(features[:, [1]])
Salary.fit(features[:, [2]])

features[:, [1]] = age.transform(features[:, [1]])
features[:, [2]] = Salary.transform(features[:, [2]])

features

from sklearn.preprocessing import OneHotEncoder

oh = OneHotEncoder(sparse_output=False)
Country = oh.fit_transform(features[:, [0]])

Country

final_set = np.concatenate((Country, features[:, [1, 2]]), axis=1)
final_set

from sklearn.preprocessing import StandardScaler
```

```
sc = StandardScaler()
sc.fit(final_set)

feat_standard_scaler = sc.transform(final_set)
feat_standard_scaler

from sklearn.preprocessing import MinMaxScaler

mms = MinMaxScaler(feature_range=(0, 1))
mms.fit(final_set)

feat_minmax_scaler = mms.transform(final_set)
feat_minmax_scaler
```

OUTPUT:

```
  df.Country.fillna(df.Country.mode()[0], inplace=True)
array([[1.        , 0.        , 0.        , 0.73913043, 0.68571429],
       [0.        , 0.        , 1.        , 0.        , 0.        ],
       [0.        , 1.        , 0.        , 0.13043478, 0.17142857],
       [0.        , 0.        , 1.        , 0.47826087, 0.37142857],
       [0.        , 1.        , 0.        , 0.56521739, 0.45079365],
       [1.        , 0.        , 0.        , 0.34782609, 0.28571429],
       [0.        , 0.        , 1.        , 0.51207729, 0.11428571],
       [1.        , 0.        , 0.        , 0.91304348, 0.88571429],
       [1.        , 0.        , 0.        , 1.        , 1.        ],
       [1.        , 0.        , 0.        , 0.43478261, 0.54285714]])
```

RESULT:

Thus,the given data was analyzed successfully and required output was obtained.

Exp No:15 Experiment to understand the data preprocessing in Data science

AIM:

To understand the data preprocessing in data science.

CODE:

```
from google.colab import files
files.upload()
import numpy as np
import pandas as pd

# Load dataset
df = pd.read_csv("/content/pre-process_datasample.csv")
print(df)

# Display information
df.info()

# Find mode of Country
print(df.Country.mode())
print(df.Country.mode()[0])
print(type(df.Country.mode()))

# Fill missing values
df.Country.fillna(df.Country.mode()[0], inplace=True)
df.Age.fillna(df.Age.median(), inplace=True)
df.Salary.fillna(round(df.Salary.mean()), inplace=True)

print(df)

# One-hot encoding of Country
print(pd.get_dummies(df.Country))

# Create updated dataset
updated_dataset = pd.concat(
   [pd.get_dummies(df.Country), df.iloc[:, [1, 2, 3]]],
   axis=1
)

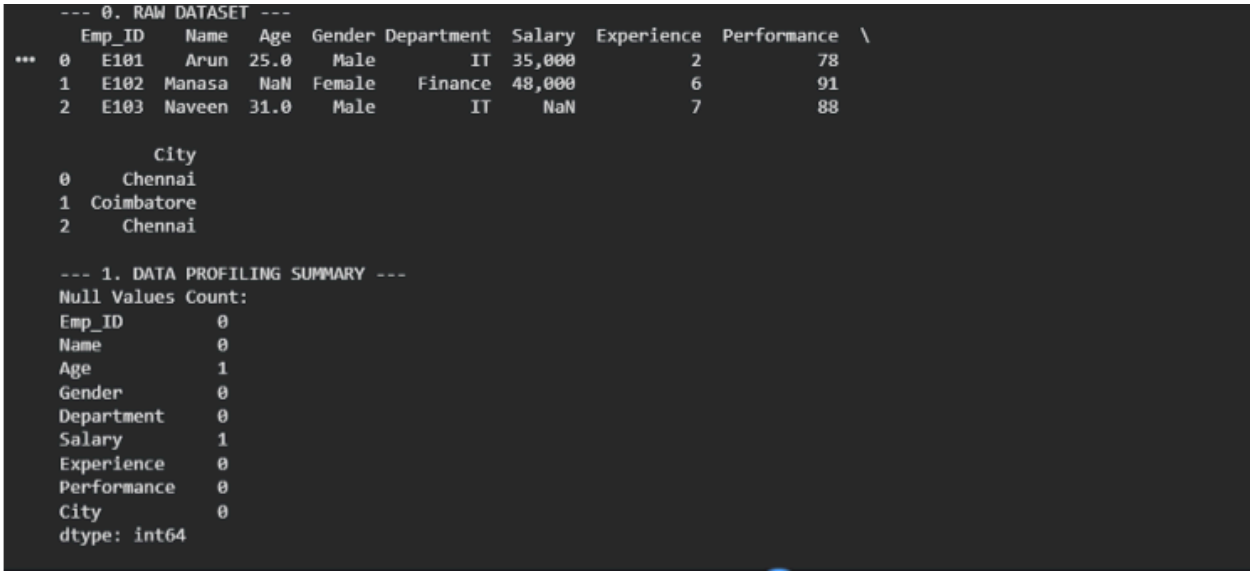
print(updated_dataset)

# Convert Purchased values to 0 and 1
updated_dataset.Purchased.replace(
   ['No', 'Yes'],
```

```
    [0, 1],
    inplace=True
)

print(updated_dataset)
```

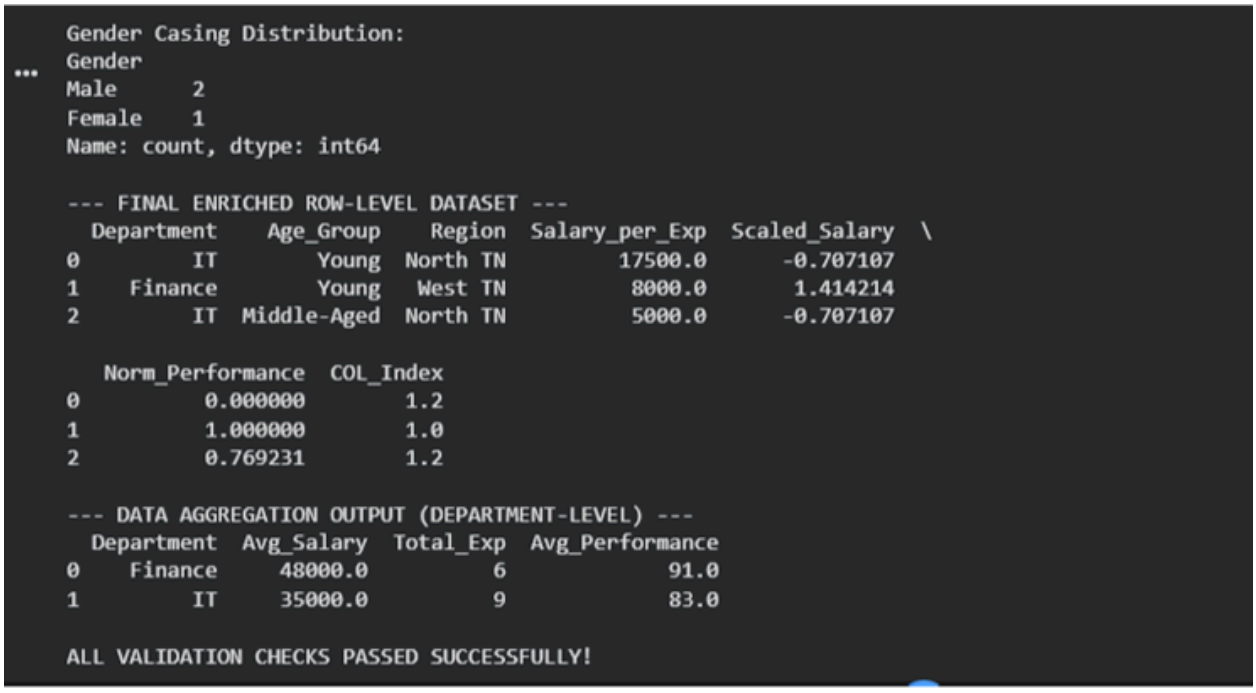
OUTPUT:



```
--- 0. RAW DATASET ---
  Emp_ID    Name   Age  Gender Department  Salary  Experience  Performance  \
0   E101    Arun  25.0    Male         IT  35,000           2           78
1   E102  Manasa   NaN  Female    Finance  48,000           6           91
2   E103  Naveen  31.0    Male         IT     NaN           7           88

         City
0     Chennai
1  Coimbatore
2     Chennai

--- 1. DATA PROFILING SUMMARY ---
Null Values Count:
Emp_ID         0
Name           0
Age            1
Gender         0
Department     0
Salary         1
Experience     0
Performance    0
City           0
dtype: int64
```



```
Gender Casing Distribution:
Gender
Male      2
Female    1
Name: count, dtype: int64

--- FINAL ENRICHED ROW-LEVEL DATASET ---
  Department    Age_Group    Region  Salary_per_Exp  Scaled_Salary  \
0         IT        Young  North TN         17500.0      -0.707107
1    Finance        Young   West TN          8000.0       1.414214
2         IT  Middle-Aged  North TN          5000.0      -0.707107

   Norm_Performance  COL_Index
0          0.000000        1.2
1          1.000000        1.0
2          0.769231        1.2

--- DATA AGGREGATION OUTPUT (DEPARTMENT-LEVEL) ---
  Department  Avg_Salary  Total_Exp  Avg_Performance
0    Finance     48000.0          6             91.0
1         IT     35000.0          9             83.0

ALL VALIDATION CHECKS PASSED SUCCESSFULLY!
```

RESULT:

Thus,the given data was analyzed successfully and required output was obtained.

EDA - QUANTITATIVE AND QUALITATIVE ANALYSIS

AIM

To understand the importance of Exploratory Data Analysis (EDA) by performing quantitative and qualitative analysis on the 'tips' data set.

PROGRAM

```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

tips = sns.load_dataset('tips')
print(tips.head())

# ---------- Quantitative analysis ----------
print(tips.describe())

sns.displot(tips.total_bill, kde=True)
plt.show()

sns.displot(tips.total_bill, kde=False)
plt.show()

sns.jointplot(x=tips.tip, y=tips.total_bill)
plt.show()

sns.jointplot(x=tips.tip, y=tips.total_bill, kind="reg")
plt.show()

sns.jointplot(x=tips.tip, y=tips.total_bill, kind="hex")
plt.show()

sns.pairplot(tips, height=2)
plt.show()

print(tips.time.value_counts())
sns.pairplot(tips, hue='time', height=2)
plt.show()

sns.pairplot(tips, hue='day', height=2)
plt.show()

sns.heatmap(tips.corr(numeric_only=True), annot=True)
plt.show()

sns.boxplot(y=tips.total_bill)
plt.show()

sns.boxplot(y=tips.tip)
plt.show()

# ---------- Qualitative analysis ----------
print(tips.day.value_counts())
print(tips.sex.value_counts())

sns.countplot(x='day', data=tips)
plt.show()
```

```
sns.countplot(x='sex', data=tips)
plt.show()

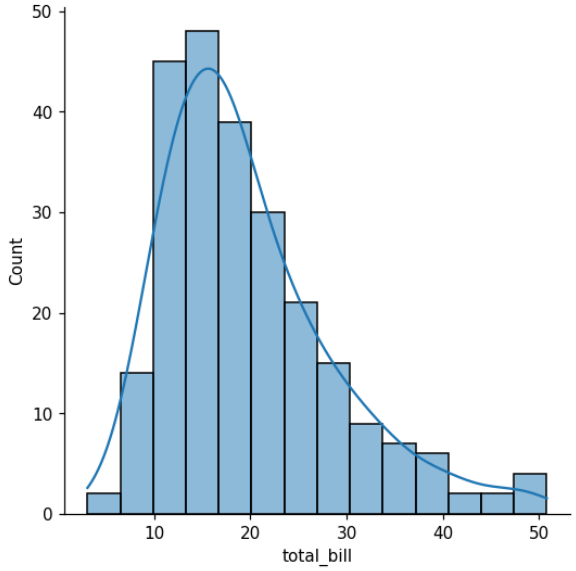
tips.sex.value_counts().plot(kind='pie', autopct='%1.1f%%')
plt.show()

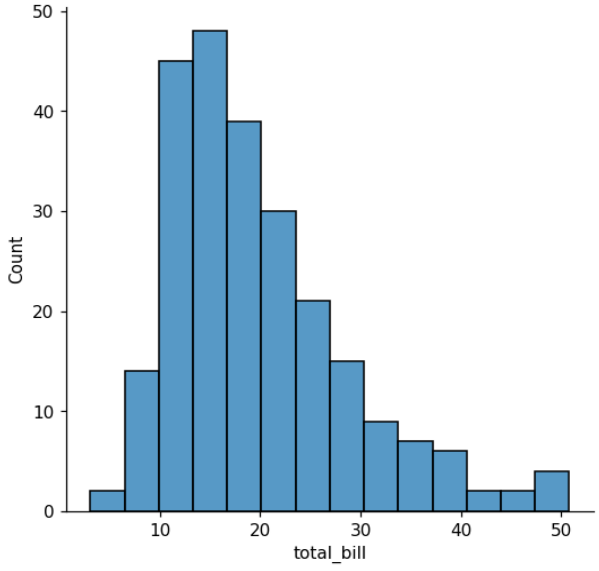
tips.sex.value_counts().plot(kind='bar')
plt.show()

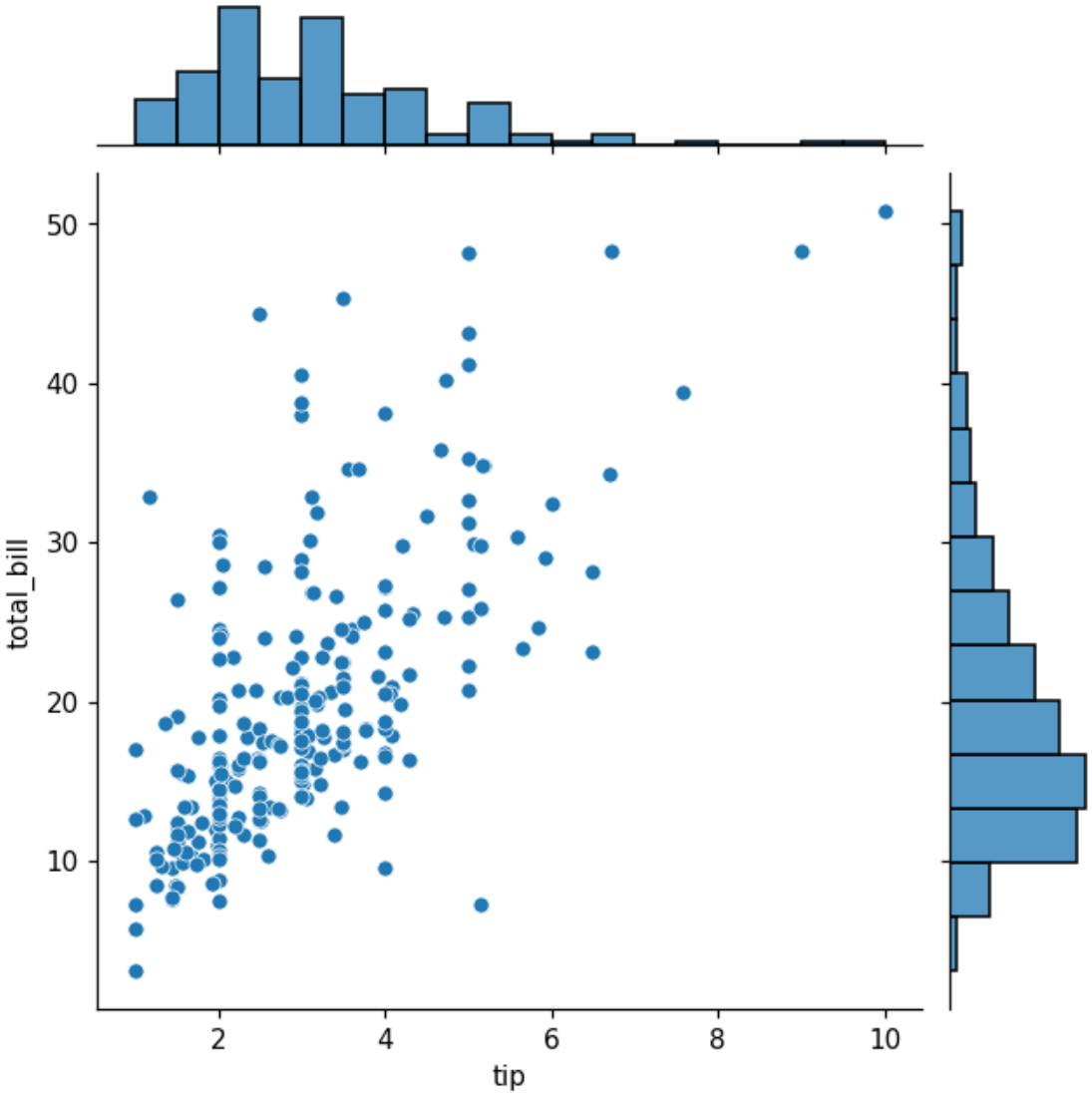
sns.countplot(x='day', data=tips[tips.time == 'Dinner'])
plt.title('Dinner count per day')
plt.show()
```

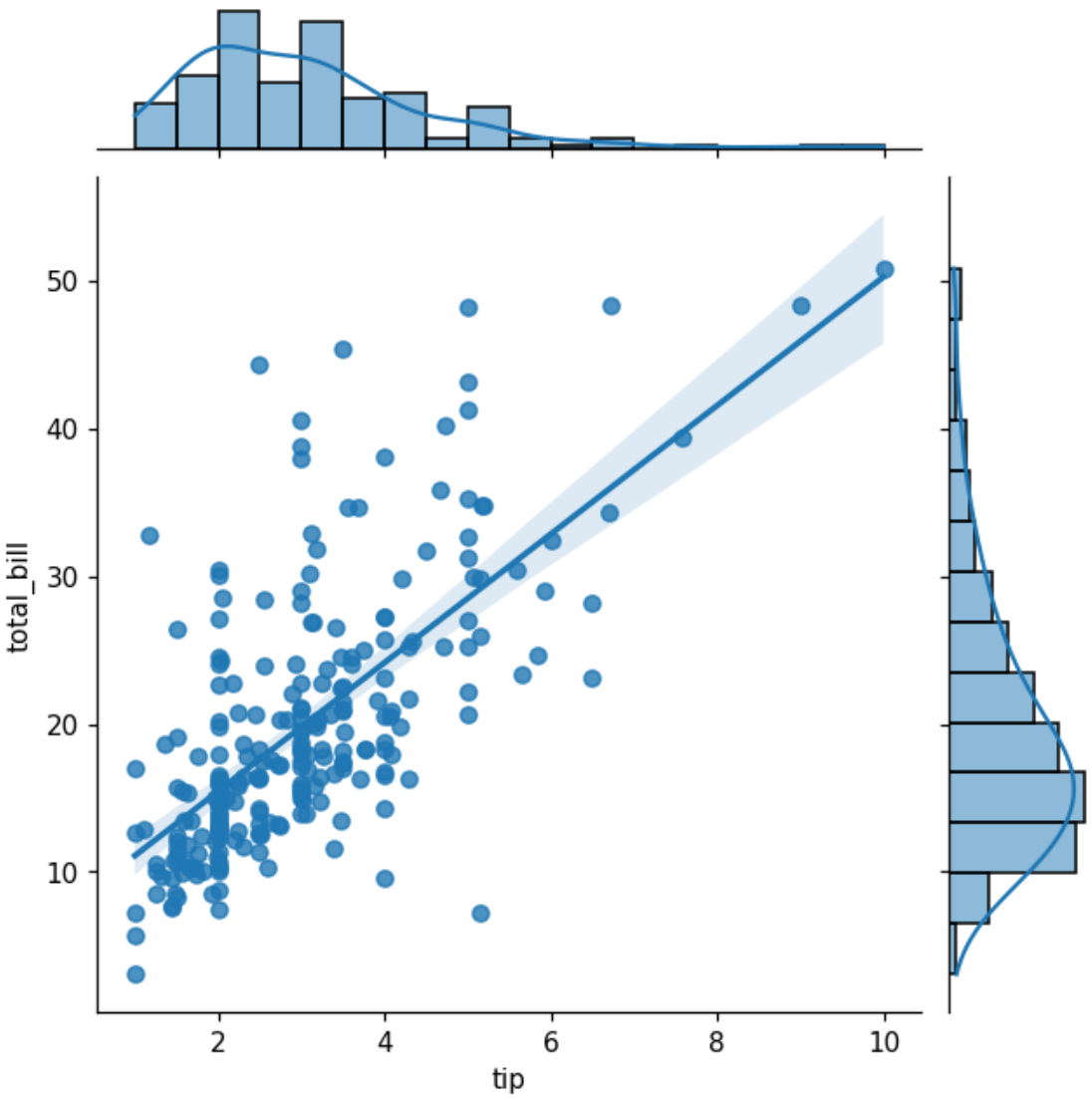
OUTPUT

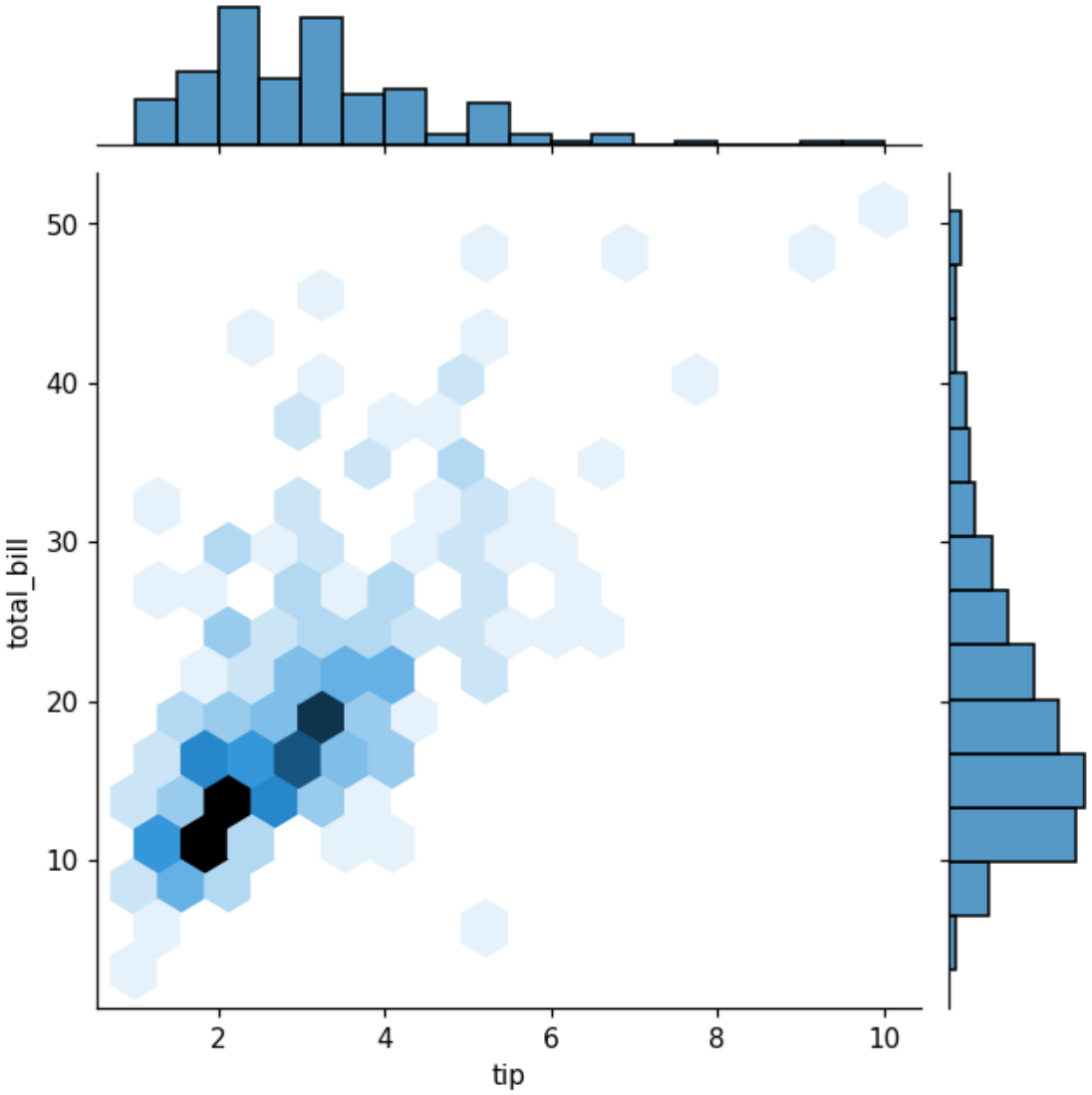
```
   total_bill   tip     sex smoker  day    time  size
0       16.99  1.01  Female     No  Sun  Dinner     2
1       10.34  1.66    Male     No  Sun  Dinner     3
2       21.01  3.50    Male     No  Sun  Dinner     3
3       23.68  3.31    Male     No  Sun  Dinner     2
4       24.59  3.61  Female     No  Sun  Dinner     4
       total_bill         tip        size
count  244.000000  244.000000  244.000000
mean    19.785943    2.998279    2.569672
std      8.902412    1.383638    0.951100
min      3.070000    1.000000    1.000000
25%     13.347500    2.000000    2.000000
50%     17.795000    2.900000    2.000000
75%     24.127500    3.562500    3.000000
max     50.810000   10.000000    6.000000
```

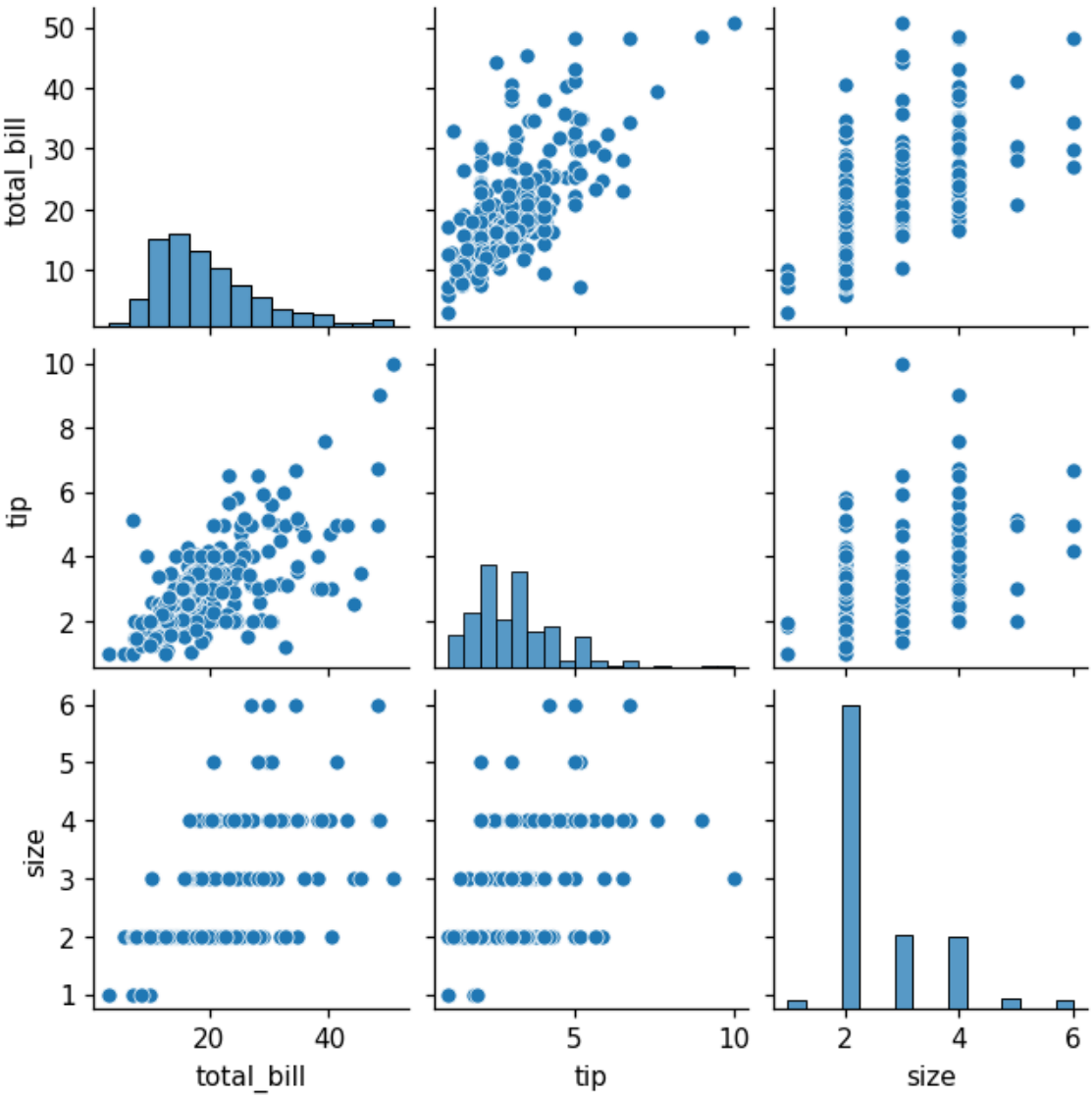




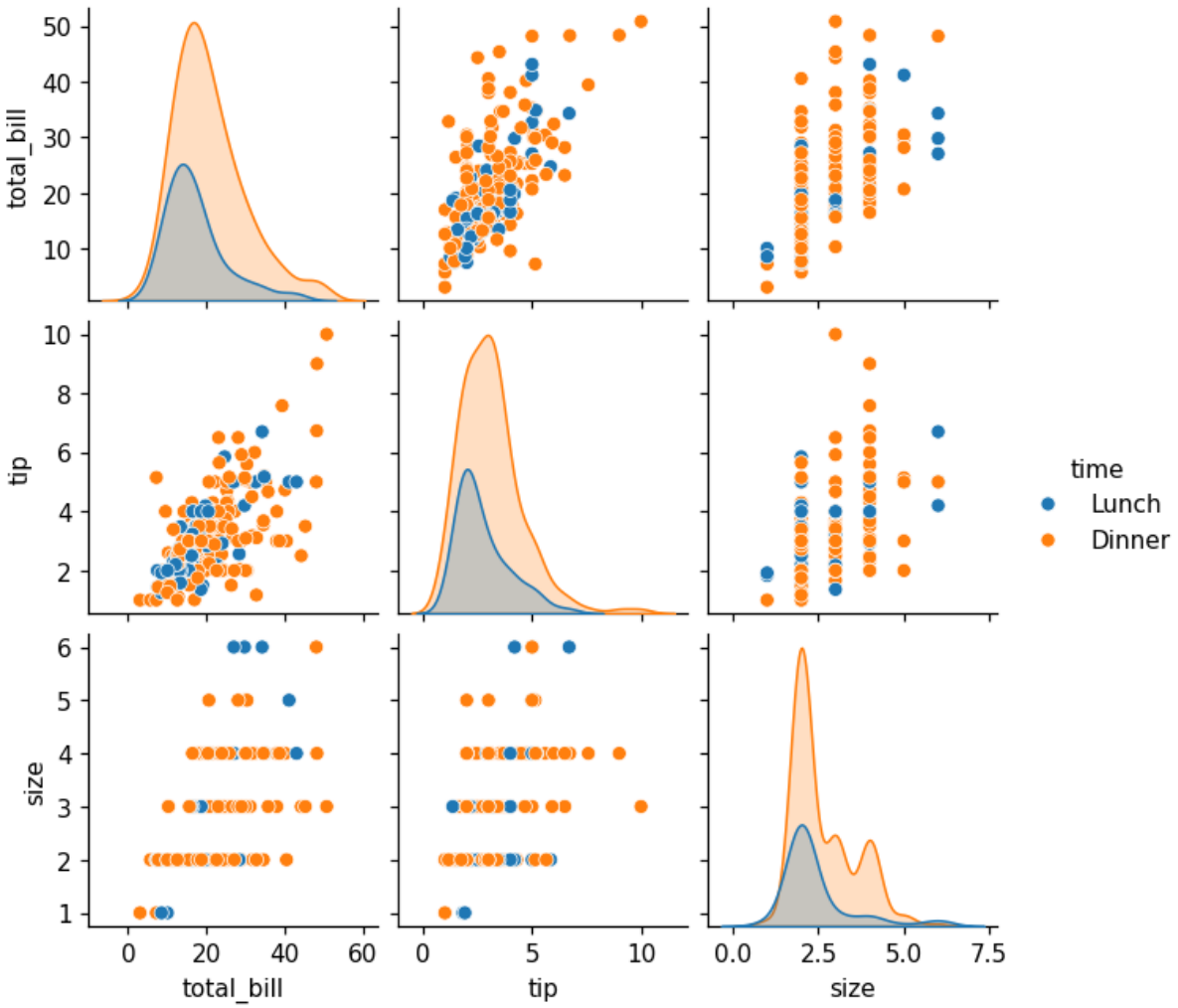

total_bill
tip
50
40
30
20
10
2
4
6
8
10

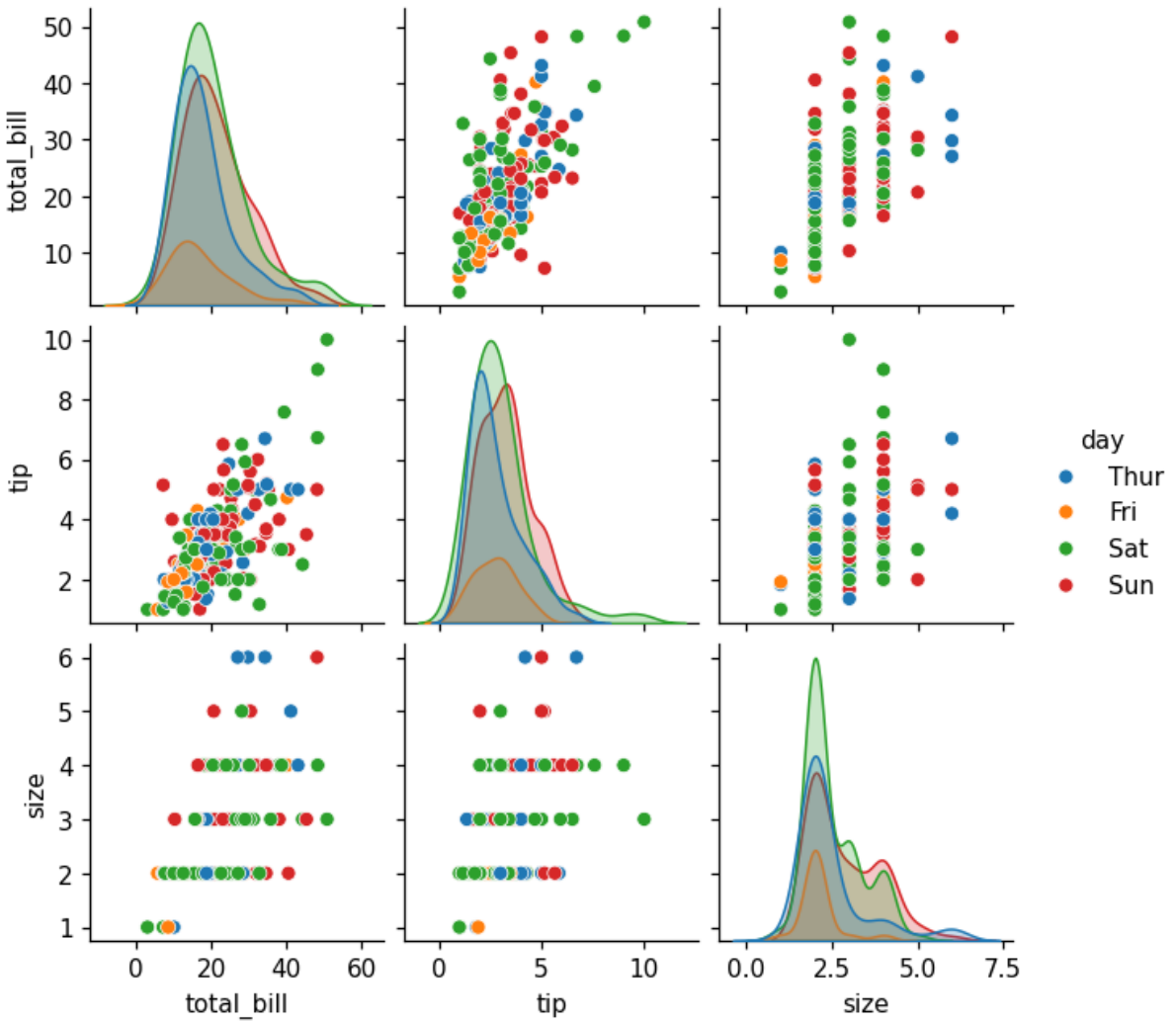

total_bill
tip
10
20
30
40
50
2
4
6
8
10


50
40
30
20
10
total_bill
2
4
6
8
10
tip

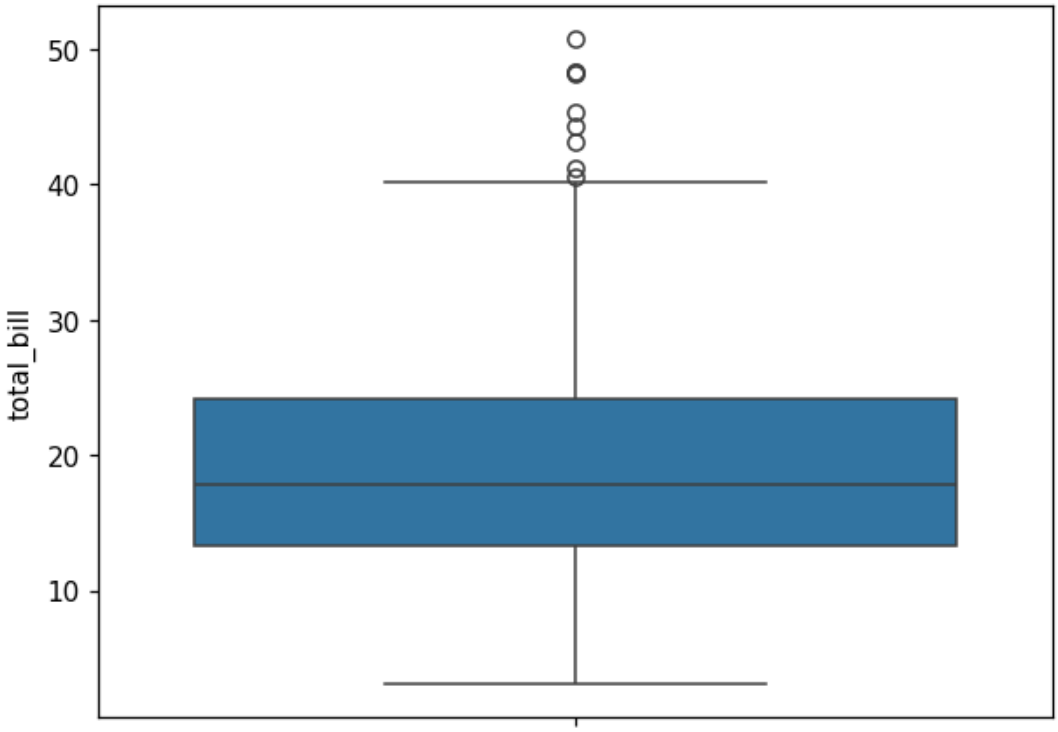


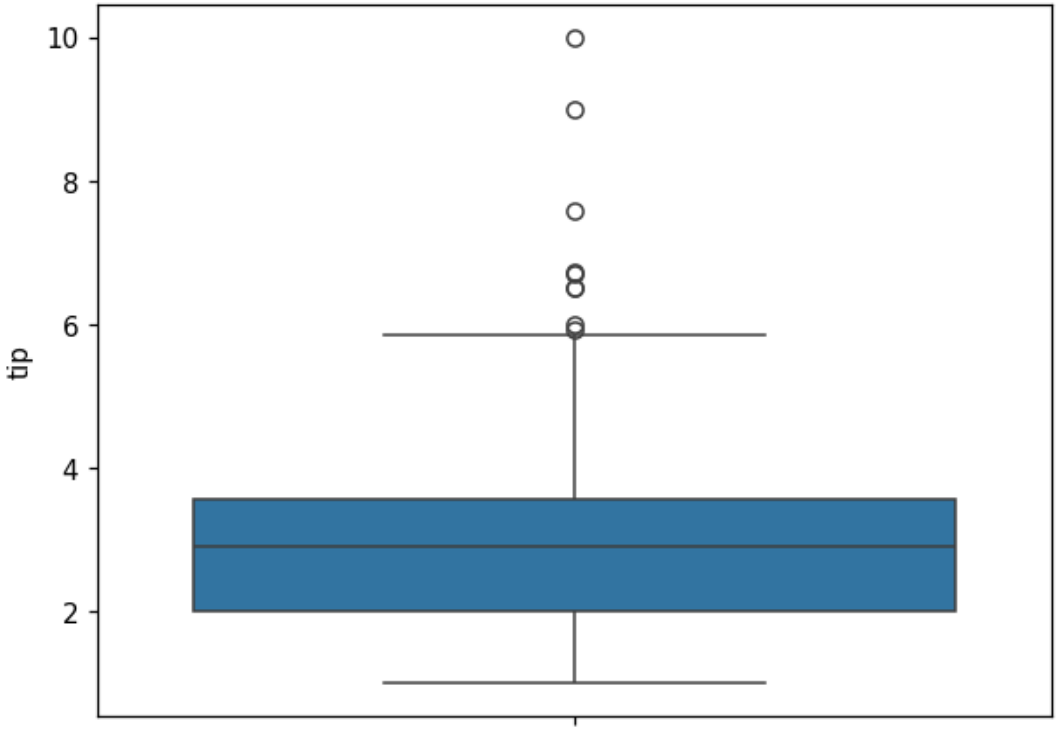

```
time
Dinner    176
Lunch      68
Name: count, dtype: int64
```


total_bill
tip
size
time
Lunch
Dinner
0
20
40
60
0
5
10
0.0
2.5
5.0
7.5
1
2
3
4
5
6
2
4
6
8
10
10
20
30
40
50

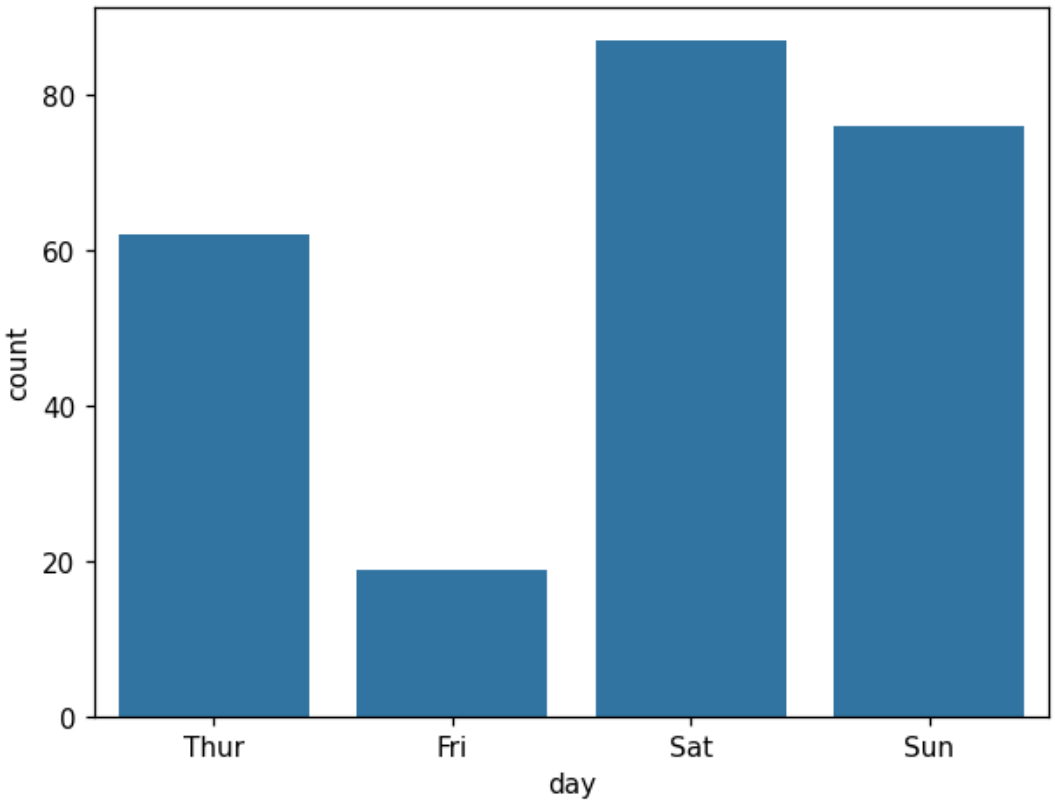

total_bill
tip
size
day
Thur
Fri
Sat
Sun
0
20
40
60
0
5
10
0.0
2.5
5.0
7.5
1
2
3
4
5
6
8
10
30
50

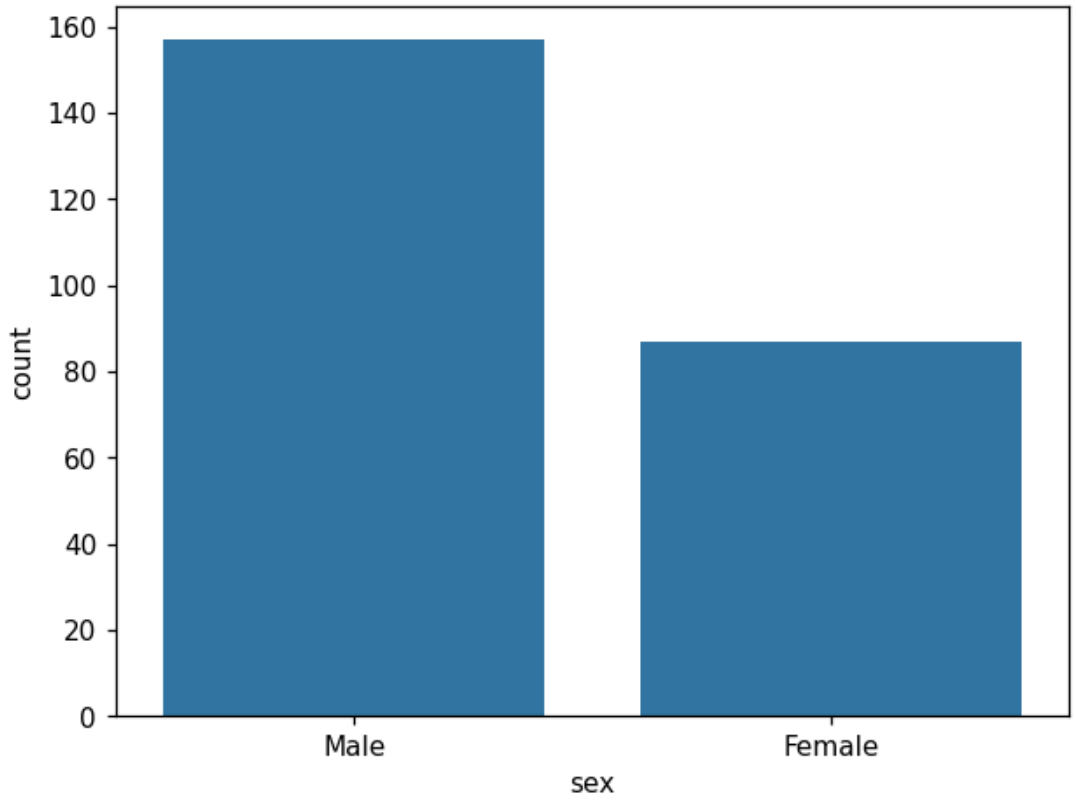

total_bill
tip
size
1
0.68
0.6
0.68
1
0.49
0.6
0.49
1
1.0
0.9
0.8
0.7
0.6
0.5

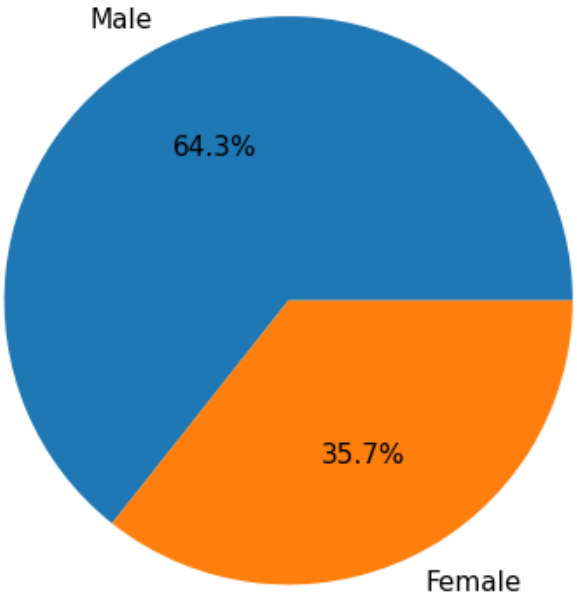


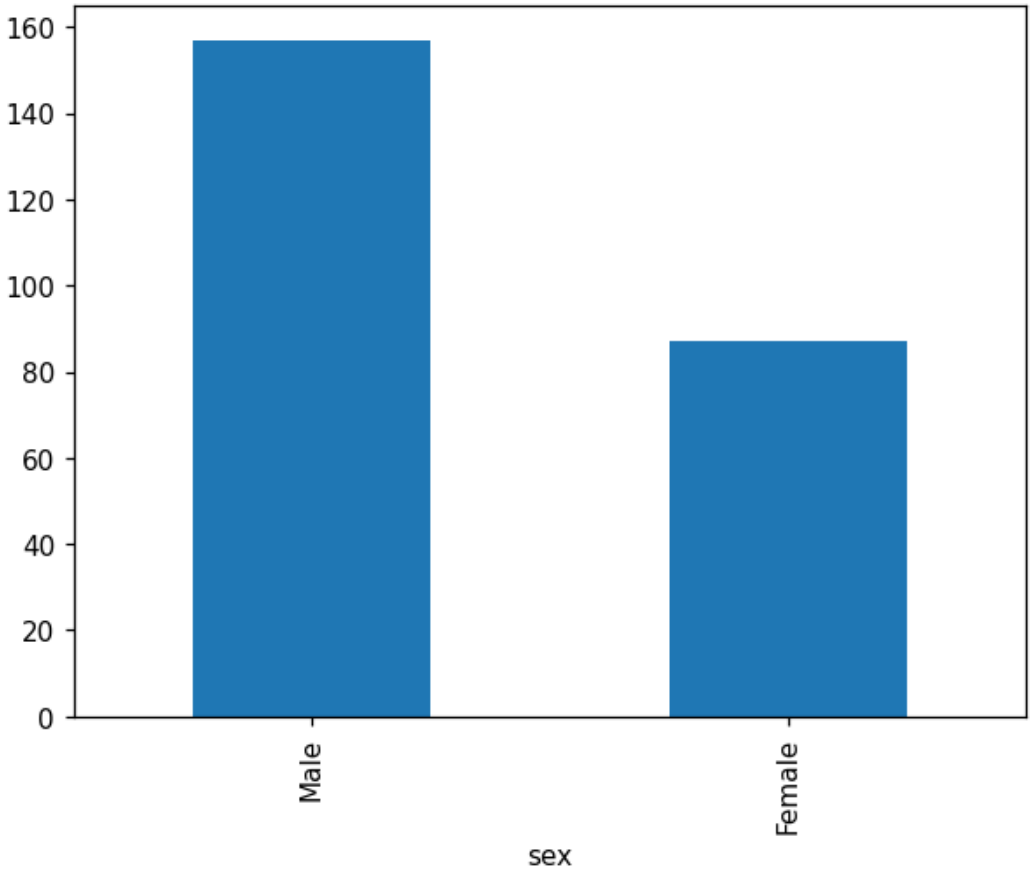


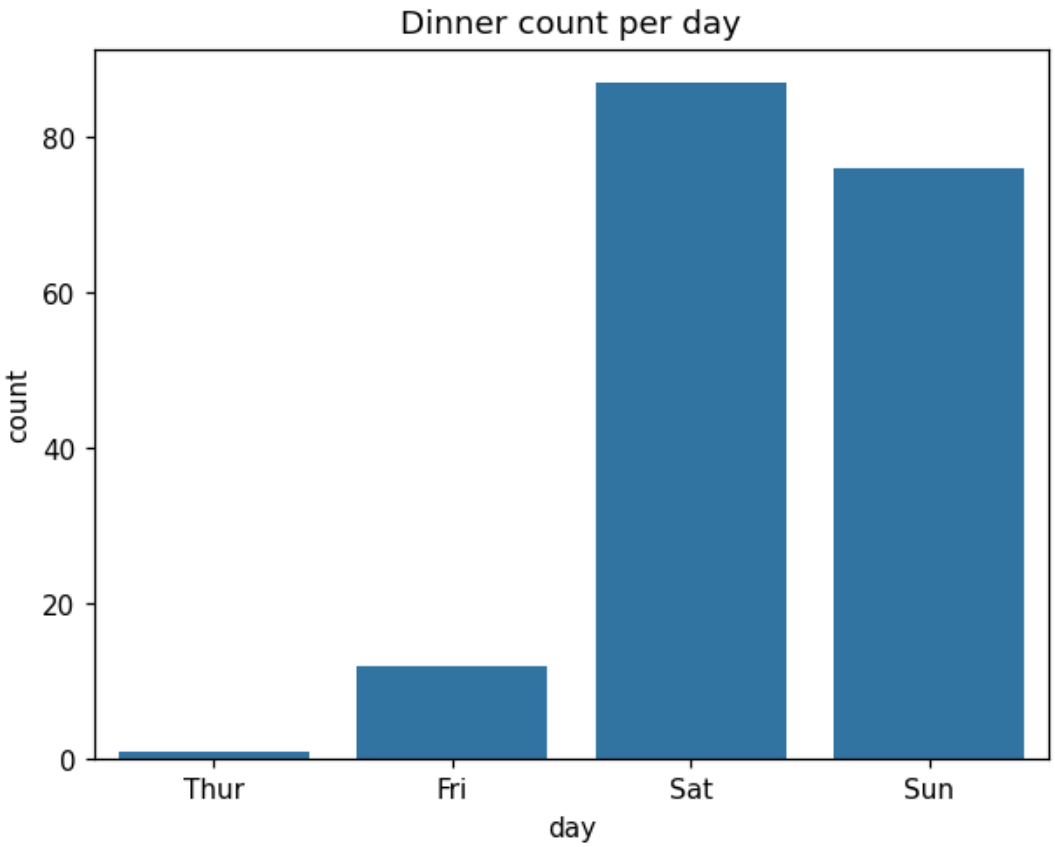

```
day
Sat     87
Sun     76
Thur    62
Fri     19
Name: count, dtype: int64
sex
Male      157
Female     87
Name: count, dtype: int64
```


80
60
40
20
0
count
Thur
Fri
Sat
Sun
day


160
140
120
100
80
60
40
20
0
count
Male
Female
sex


Male
64.3%
35.7%
Female


160
140
120
100
80
60
40
20
0
Male
Female
sex



RESULT

Thus exploratory data analysis (quantitative and qualitative) was performed on the tips data set. The total bill and tip are positively correlated, the bill amounts are right-skewed with a few high-value outliers, and most of the records belong to dinner time.

LINEAR REGRESSION

AIM

To understand Linear Regression by building a model that predicts the salary of an employee from the years of experience.

PROGRAM

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('Salary_data.csv')
print(df)
df.info()
df.dropna(inplace=True)
df.info()
print(df.describe())

features = df.iloc[:, [0]].values
label = df.iloc[:, [1]].values

from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=23)

from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(x_train, y_train)

print("Training score :", model.score(x_train, y_train))
print("Testing score  :", model.score(x_test, y_test))
print("Coefficient    :", model.coef_)
print("Intercept      :", model.intercept_)

# Visualizing the regression line
plt.scatter(x_train, y_train, color='blue', label='Training data')
plt.scatter(x_test, y_test, color='green', label='Testing data')
plt.plot(features, model.predict(features), color='red', label='Regression line')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.title('Salary vs Experience')
plt.legend()
plt.show()

# Saving and loading the model
import pickle
pickle.dump(model, open('SalaryPred.model', 'wb'))
model = pickle.load(open('SalaryPred.model', 'rb'))

yr_of_exp = 5.0          # years of experience (input value)
yr_of_exp_NP = np.array([[yr_of_exp]])
Salary = model.predict(yr_of_exp_NP)
print("Estimated Salary for {} years of experience is {}".format(yr_of_exp, Salary))
```

OUTPUT

```
    YearsExperience  Salary
0               1.1   39343
1               1.3   46205
2               1.5   37731
```

```
3               2.0   43525
4               2.2   39891
5               2.9   56642
6               3.0   60150
7               3.2   54445
8               3.2   64445
9               3.7   57189
10              3.9   63218
11              4.0   55794
12              4.0   56957
13              4.1   57081
14              4.5   61111
15              4.9   67938
16              5.1   66029
17              5.3   83088
18              5.9   81363
19              6.0   93940
20              6.8   91738
21              7.1   98273
22              7.9  101302
23              8.2  113812
24              8.7  109431
25              9.0  105582
26              9.5  116969
27              9.6  112635
28             10.3  122391
29             10.5  121872
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
       YearsExperience         Salary
count        30.000000      30.000000
mean          5.313333   76003.000000
std           2.837888   27414.429785
min           1.100000   37731.000000
25%           3.200000   56720.750000
50%           4.700000   65237.000000
75%           7.700000  100544.750000
max          10.500000  122391.000000
Training score : 0.9603182547438908
Testing score  : 0.9184170849214232
Coefficient    : [[9281.30847068]]
Intercept      : [27166.73682891]
```




```
Estimated Salary for 5.0 years of experience is [[73573.27918232]]
```

RESULT

Thus a Linear Regression model was built to predict the salary from the years of experience. The model fits the data well (training score 0.96, testing score 0.92) and was used to predict the salary for a given experience.

K-NEAREST NEIGHBOURS (KNN) ALGORITHM

AIM

To understand the KNN algorithm by classifying the species of the Iris flowers.

PROGRAM

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('Iris.csv')
df.info()
print(df.variety.value_counts())
print(df.head())

features = df.iloc[:, :-1].values
label = df.iloc[:, 4].values

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
xtrain, xtest, ytrain, ytest = train_test_split(
    features, label, test_size=.2, random_state=23)

model_KNN = KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain, ytrain)

print("Training score :", model_KNN.score(xtrain, ytrain))
print("Testing score  :", model_KNN.score(xtest, ytest))

from sklearn.metrics import confusion_matrix, classification_report
ypred = model_KNN.predict(xtest)
print("\nConfusion matrix:\n", confusion_matrix(ytest, ypred))
print("\nClassification report:\n", classification_report(ytest, ypred))

# Predicting a new sample
sample = np.array([[5.1, 3.5, 1.4, 0.2]])
print("Prediction for", sample[0], "->", model_KNN.predict(sample)[0])

# Effect of K on accuracy
scores = []
for k in range(1, 16):
    knn = KNeighborsClassifier(n_neighbors=k).fit(xtrain, ytrain)
    scores.append(knn.score(xtest, ytest))
plt.plot(range(1, 16), scores, marker='o')
plt.xlabel('K (number of neighbours)')
plt.ylabel('Testing accuracy')
plt.title('Accuracy for different values of K')
plt.show()
```

OUTPUT

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    str
```

```
dtypes: float64(4), str(1)
memory usage: 6.0 KB
variety
Setosa        50
Versicolor    50
Virginica     50
Name: count, dtype: int64
   sepal.length  sepal.width  petal.length  petal.width variety
0           5.1          3.5           1.4          0.2  Setosa
1           4.9          3.0           1.4          0.2  Setosa
2           4.7          3.2           1.3          0.2  Setosa
3           4.6          3.1           1.5          0.2  Setosa
4           5.0          3.6           1.4          0.2  Setosa
Training score : 0.9583333333333334
Testing score  : 1.0

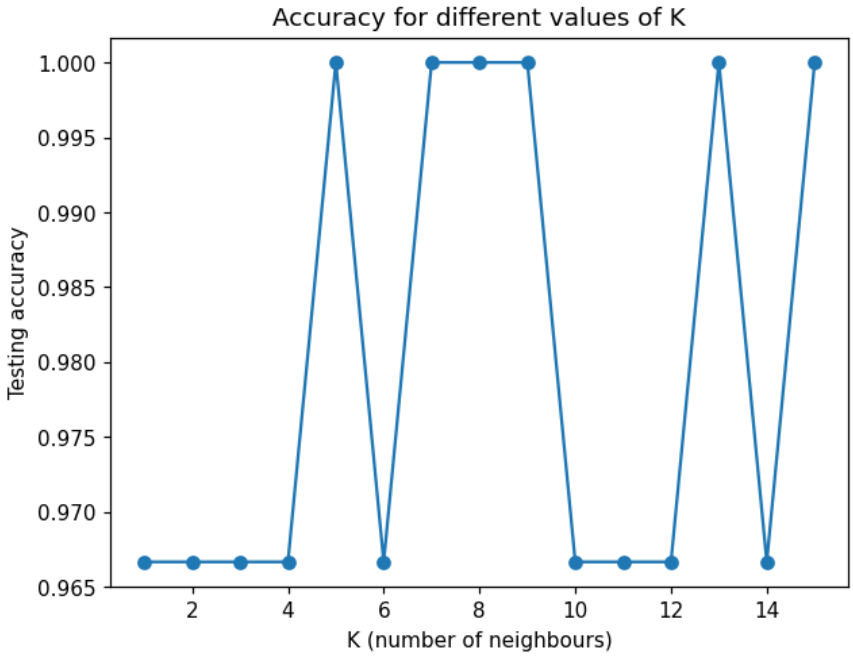
Confusion matrix:
 [[12  0  0]
 [ 0  8  0]
 [ 0  0 10]]

Classification report:
               precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        12
  Versicolor       1.00      1.00      1.00         8
   Virginica       1.00      1.00      1.00        10

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30

Prediction for [5.1 3.5 1.4 0.2] -> Setosa
```



RESULT

Thus the KNN algorithm was implemented on the Iris data set and the species of the flowers were classified with high accuracy.

LOGISTIC REGRESSION

AIM

To understand the Logistic Regression algorithm by classifying tumours as malignant or benign for a given data set.

PROGRAM

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# The sigmoid (logistic) function used by Logistic Regression
z = np.linspace(-8, 8, 200)
plt.plot(z, 1 / (1 + np.exp(-z)))
plt.axhline(0.5, color='red', linestyle='--')
plt.title('Sigmoid function')
plt.xlabel('z')
plt.ylabel('probability')
plt.show()

# Load the data set
data = load_breast_cancer(as_frame=True)
df = data.frame
print("Shape :", df.shape)
print(df.iloc[:, [0, 1, 2, 3, -1]].head())
print("Missing values :", df.isnull().sum().sum())
print(df['target'].value_counts())

features = df.drop('target', axis=1)
label = df['target']
x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=23)

# Feature scaling
sc = StandardScaler()
x_train = sc.fit_transform(x_train)
x_test = sc.transform(x_test)

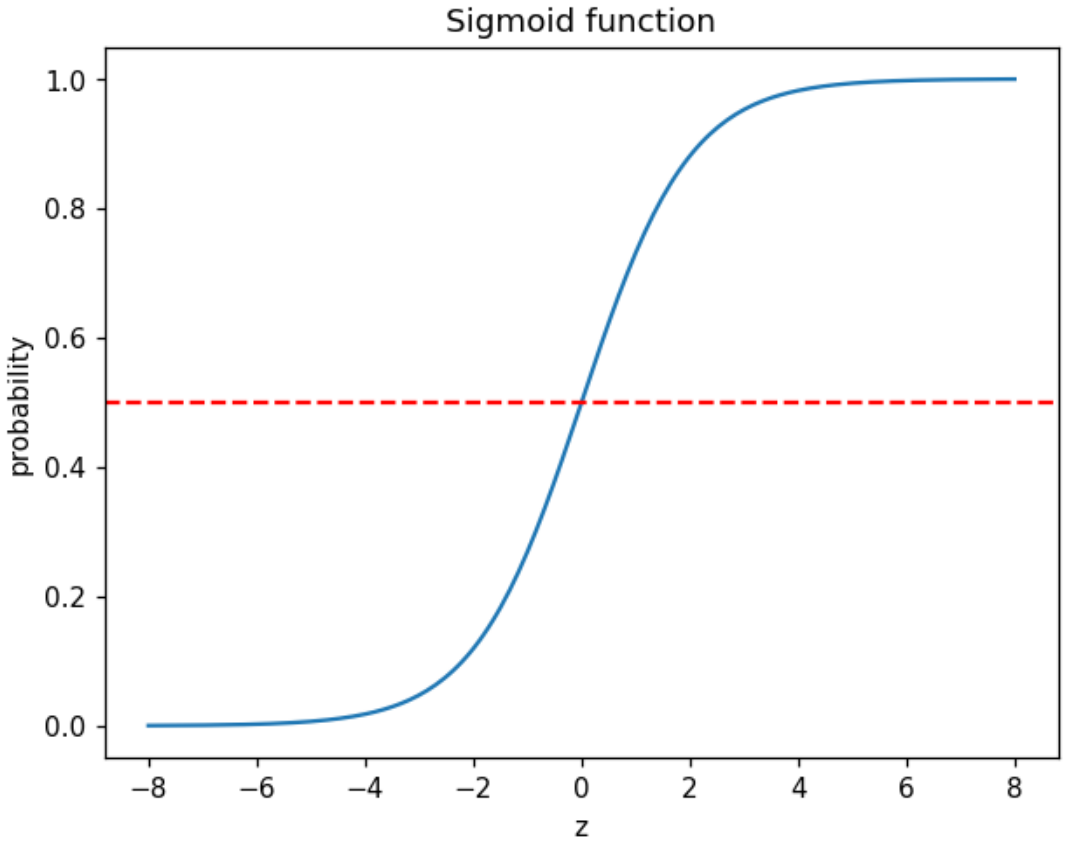
# Training the model
model = LogisticRegression(max_iter=1000)
model.fit(x_train, y_train)

# Prediction and evaluation
y_pred = model.predict(x_test)
print("Training score :", model.score(x_train, y_train))
print("Testing accuracy:", accuracy_score(y_test, y_pred))
print("Probabilities of first 5 test records [malignant, benign]:")
print(model.predict_proba(x_test[:5]).round(4))
print("Predicted classes :", y_pred[:5])
print("Actual classes    :", y_test.values[:5])

cm = confusion_matrix(y_test, y_pred)
print("\nConfusion matrix:\n", cm)
print("\nClassification report:")
print(classification_report(y_test, y_pred, target_names=data.target_names))
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=data.target_names, yticklabels=data.target_names)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion matrix')
plt.show()
```

OUTPUT



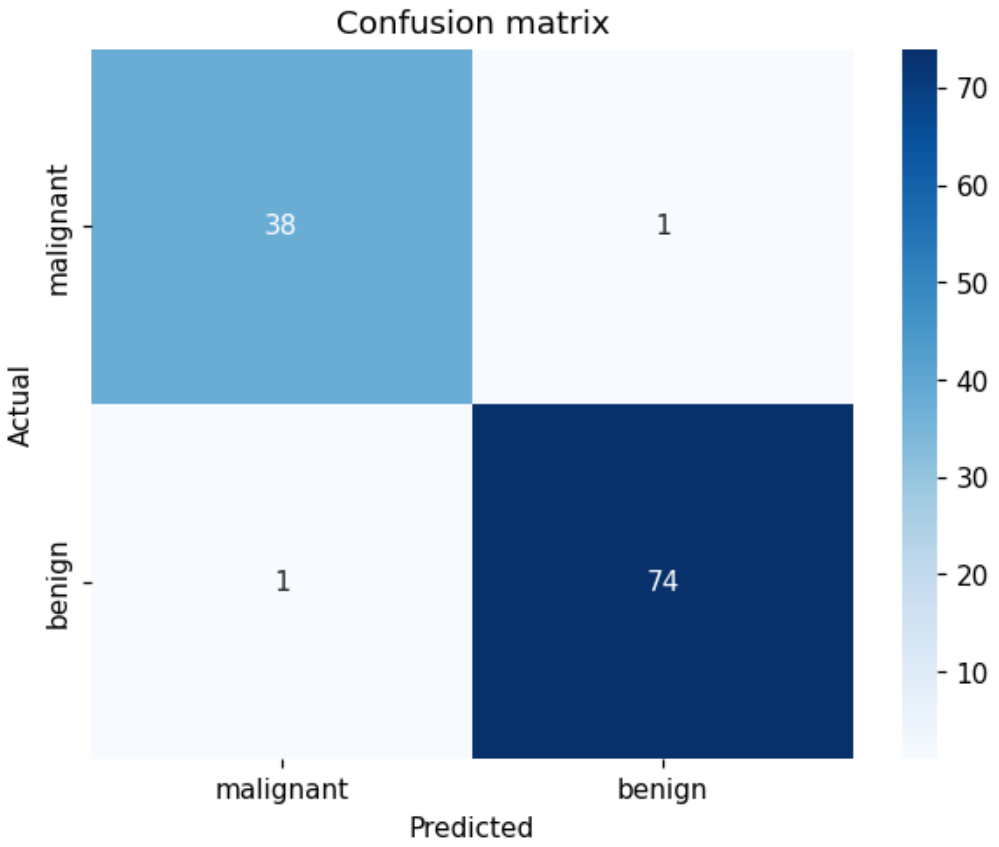

```
Shape : (569, 31)
   mean radius  mean texture  mean perimeter  mean area  target
0        17.99         10.38          122.80     1001.0       0
1        20.57         17.77          132.90     1326.0       0
2        19.69         21.25          130.00     1203.0       0
3        11.42         20.38           77.58      386.1       0
4        20.29         14.34          135.10     1297.0       0
Missing values : 0
target
1    357
0    212
Name: count, dtype: int64
Training score : 0.989010989010989
Testing accuracy: 0.9824561403508771
Probabilities of first 5 test records [malignant, benign]:
[[0.0044 0.9956]
 [0.5369 0.4631]
 [1.     0.    ]
 [0.0336 0.9664]
 [0.9983 0.0017]]
Predicted classes : [1 0 0 1 0]
Actual classes    : [1 1 0 1 0]

Confusion matrix:
 [[38  1]
 [ 1 74]]
```

```
Classification report:
              precision    recall  f1-score   support

   malignant       0.97      0.97      0.97        39
      benign       0.99      0.99      0.99        75

    accuracy                           0.98       114
   macro avg       0.98      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114
```



RESULT

Thus the Logistic Regression model was built and trained on the Breast Cancer data set and it classified the tumours with high accuracy on the test data.

K-MEANS CLUSTERING ALGORITHM

AIM

To understand the K-Means clustering algorithm by grouping the Iris flowers into clusters for a given data set.

PROGRAM

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

df = pd.read_csv('Iris.csv')
print(df.head())
features = df.iloc[:, :-1].values        # label is not used for clustering

# Elbow method
wcss = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=23)
    km.fit(features)
    wcss.append(km.inertia_)
    print(f"K = {k:<3} WCSS = {km.inertia_:.2f}")

plt.plot(range(1, 11), wcss, marker='o')
plt.title('Elbow method')
plt.xlabel('Number of clusters (K)')
plt.ylabel('WCSS')
plt.show()

# Final model with K = 3
kmeans = KMeans(n_clusters=3, init='k-means++', n_init=10, random_state=23)
labels = kmeans.fit_predict(features)

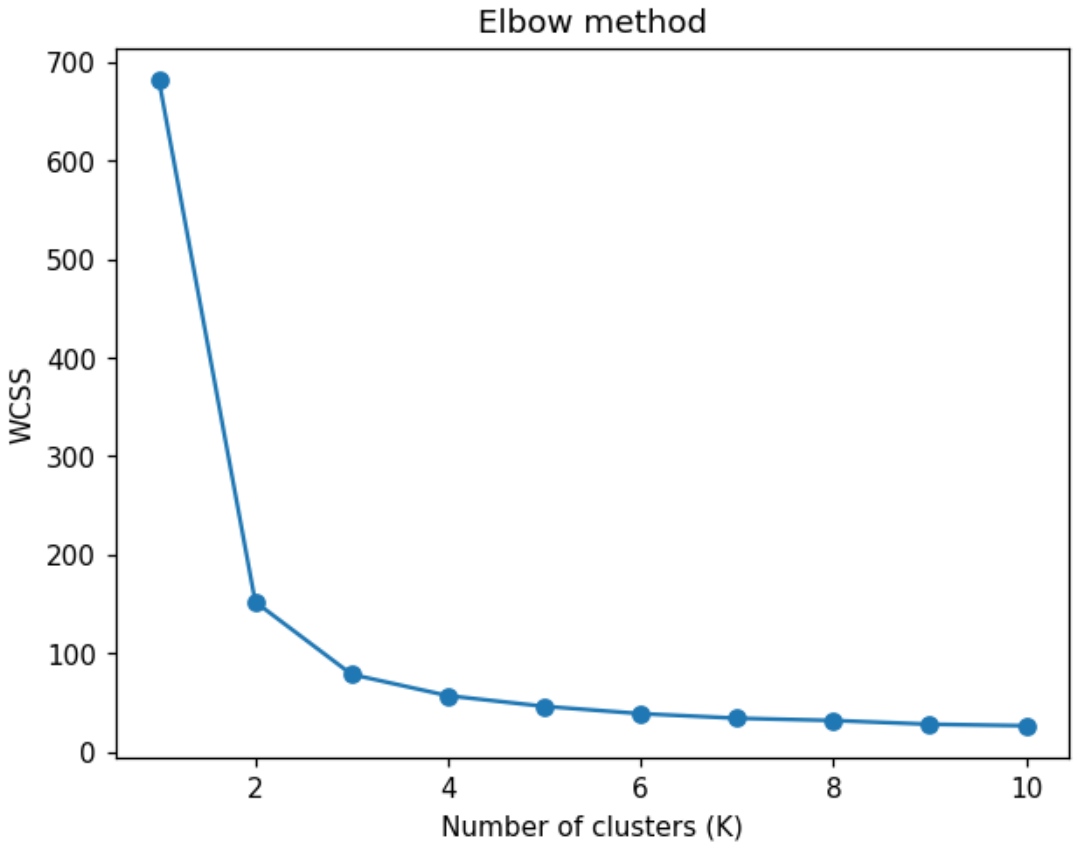
print("\nCluster centres:\n", kmeans.cluster_centers_.round(3))
print("Cluster sizes :", np.bincount(labels))
print("Silhouette score:", round(silhouette_score(features, labels), 4))

# Comparison with the actual species
print("\nClusters vs actual species:")
print(pd.crosstab(df['variety'], labels, colnames=['cluster']))

# Visualization (petal length vs petal width)
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].scatter(features[:, 2], features[:, 3], c=labels, cmap='viridis')
ax[0].scatter(kmeans.cluster_centers_[:, 2], kmeans.cluster_centers_[:, 3],
              s=200, c='red', marker='X', label='Centroids')
ax[0].set_title('K-Means clusters')
ax[0].set_xlabel('petal.length')
ax[0].set_ylabel('petal.width')
ax[0].legend()
actual = df['variety'].astype('category').cat.codes
ax[1].scatter(features[:, 2], features[:, 3], c=actual, cmap='viridis')
ax[1].set_title('Actual species')
ax[1].set_xlabel('petal.length')
ax[1].set_ylabel('petal.width')
plt.tight_layout()
plt.show()
```

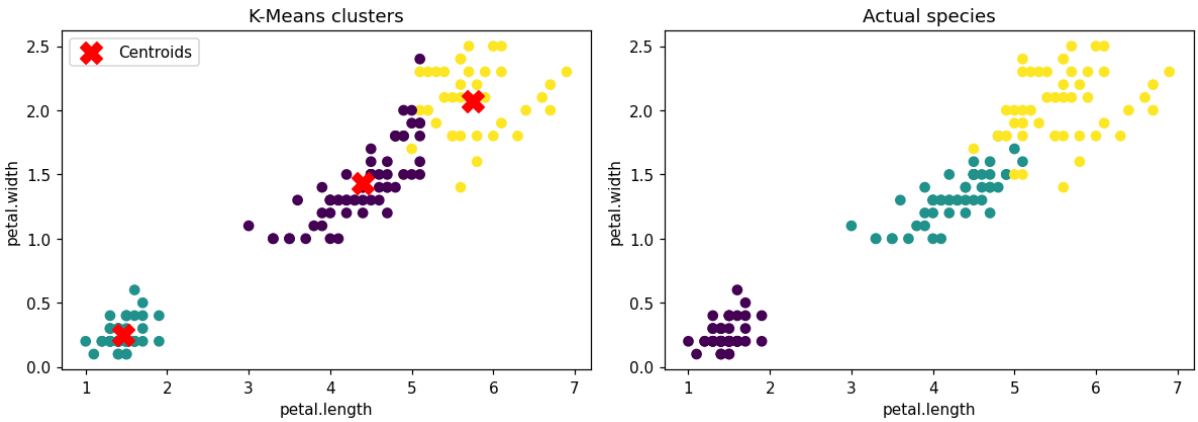
OUTPUT

```
   sepal.length  sepal.width  petal.length  petal.width variety
0           5.1          3.5           1.4          0.2  Setosa
1           4.9          3.0           1.4          0.2  Setosa
2           4.7          3.2           1.3          0.2  Setosa
3           4.6          3.1           1.5          0.2  Setosa
4           5.0          3.6           1.4          0.2  Setosa
K = 1   WCSS = 681.37
K = 2   WCSS = 152.35
K = 3   WCSS = 78.85
K = 4   WCSS = 57.23
K = 5   WCSS = 46.45
K = 6   WCSS = 39.04
K = 7   WCSS = 34.41
K = 8   WCSS = 32.10
K = 9   WCSS = 28.32
K = 10  WCSS = 26.81
```




```
Cluster centres:
 [[5.902 2.748 4.394 1.434]
 [5.006 3.428 1.462 0.246]
 [6.85  3.074 5.742 2.071]]
Cluster sizes : [62 50 38]
Silhouette score: 0.5528

Clusters vs actual species:
cluster      0   1   2
variety
Setosa       0  50   0
Versicolor  48   0   2
Virginica   14   0  36
```



RESULT

Thus the K-Means clustering algorithm was implemented on the Iris data set. The Elbow method gave K = 3 and the clusters closely match the three actual species.